cadence

INVENTIVE

Universal Verification Methodology (UVM)



September 2011 – UVM1.1

UVM Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

cadence™

What is UVM?

UVM is the Universal Verification Methodology

- A methodology and a library that codifies the best practices for efficient and exhaustive verification.

- *A complete, proven solution*
  - Proven solution, with a success record and large community of users with methodology knowledge and commitment
  - Well-thought-out solution for a wide variety of verification challenges
- *Open*
  - A key motivation for moving to SystemVerilog
  - An Accellera standard, supported by Cadence, Mentor and Synopsys
- *Enables reuse of verification environments*
  - Verification IP can dramatically speed-up delivery and improve quality
  - A common methodology that also works for other languages (SystemVerilog, ***e***, SystemC)

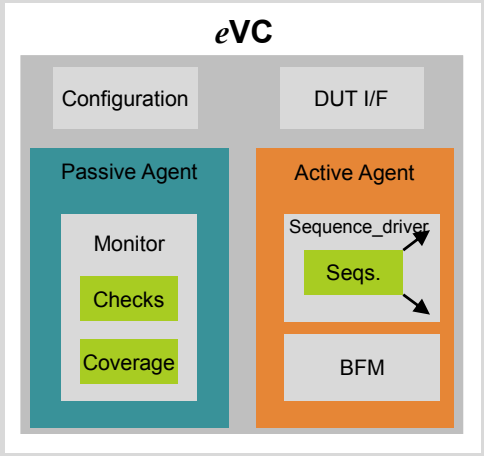
© 2011 Cadence Design Systems, Inc. All rights reserved.

cadence®

UVM is Production Ready

Cadence Verification Methodology Leadership from the Beginning…

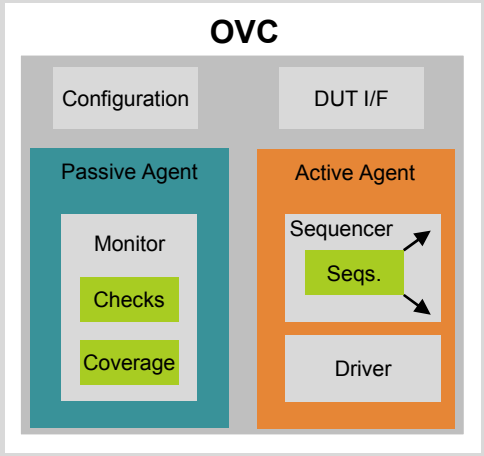
2000 *e*RM



Included:
- Architecture
- Sequences
- Messaging/logging
- Reset/clocking
- Common test interface
- Packaging & code structure

Added:
- Module to system reuse
- Register package (vr_ad)
- HW/SW co-verification

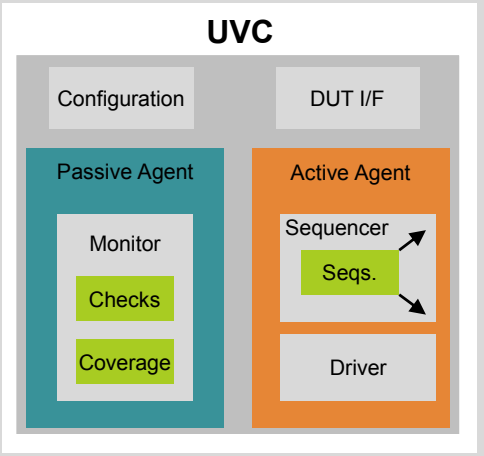
Jan 2008 OVM



Added:
- SystemVerilog support
  - Architecture
  - Sequences
  - Factory
  - Configuration mechanism
  - Field automation
  - Message macros
  - Register pkg (contribution)
- Integration of AVM base classes
- TLM Communication Channel
- Multi-language interoperability

OVM 2.1.1

Feb 2011 UVM



Added:
- SV callback enhancements
  - Report catching callback
- SV end of test enhancements
  - Closer to *e*RM capability
- Register support
- Enhanced phasing
- Partial TLM2 support
- Improved configuration and command line support
- Script for “o” to “u” migration for OVM users

© 2011 Cadence Design Systems, Inc. All rights reserved.

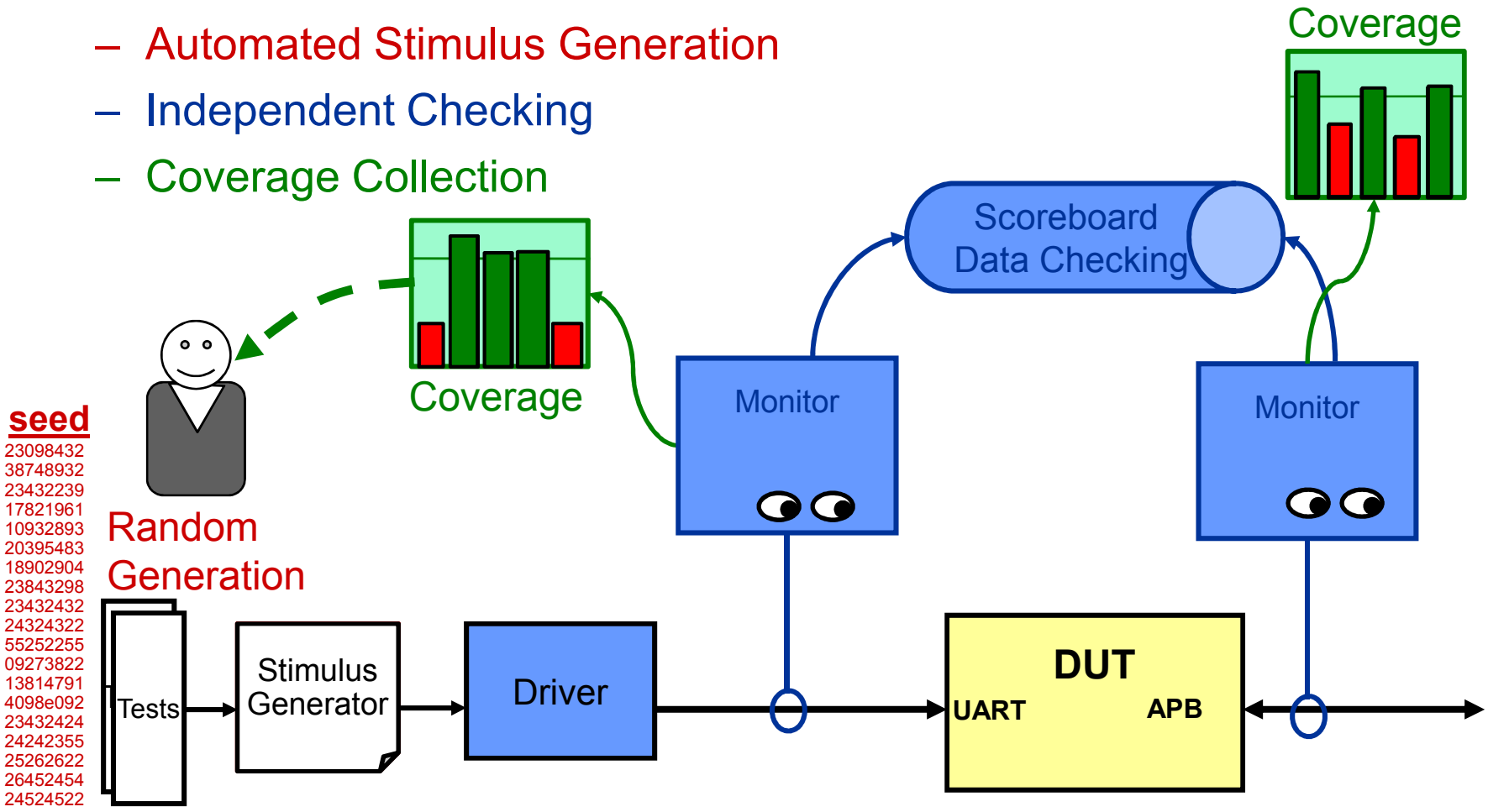
UVM and Coverage Driven Verification

- Coverage-driven verification(CDV) combines the following aspects to significantly reduce the time spent verifying a design:
  - Automatic stimulus generation
  - Self-checking testbenches
  - Coverage metrics
- Why CDV?
  - Eliminate the effort and time to write hundreds of tests
  - Ensure thorough verification using upfront goal setting
- UVM provides the framework to achieve CDV

cadence™

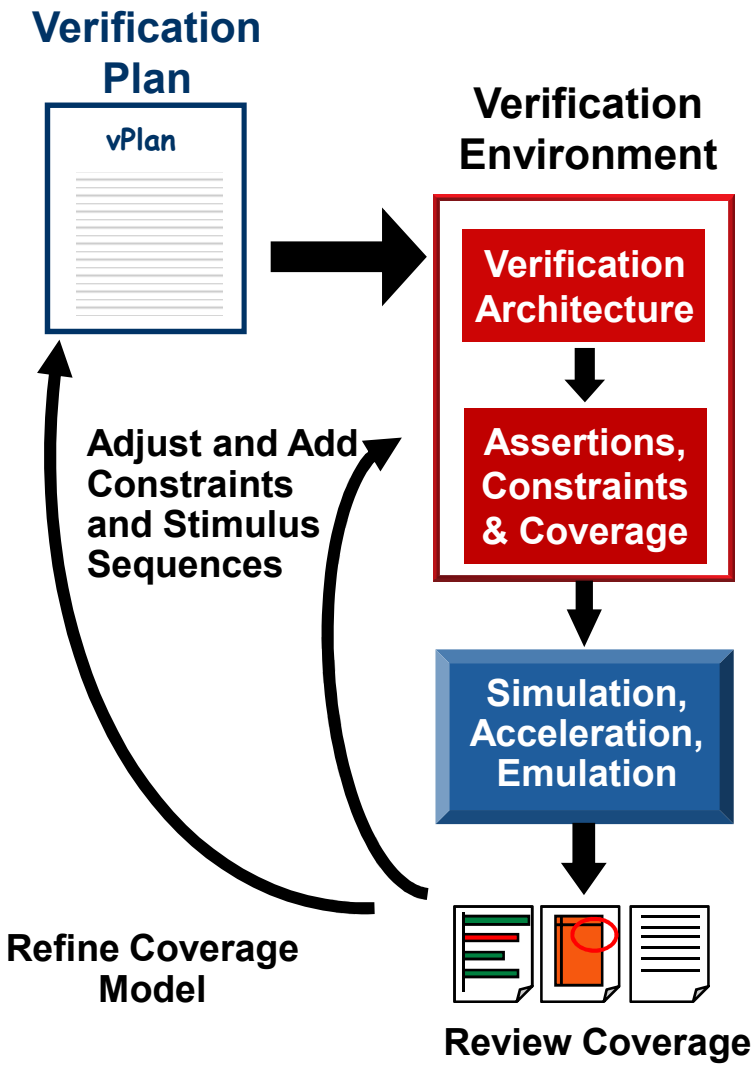
Coverage Driven Verification

- Components of a coverage driven verification environment
  - Automated Stimulus Generation
  - Independent Checking
  - Coverage Collection



cadence

Coverage Driven Verification Flow



- Create a verification plan
  - Based on specification requirements
  - Decide what to test and how
- Create smart testbench that:
  - Generates random legal stimulus
  - Includes assertions and coverage to measure progress against the goals
  - Checks for undesired behavior (BUGS!)
- Implement coverage model based on the verification plan, in place early on
- Iteratively run simulations and analyze coverage
- Adjust constraints and add new scenarios targeting coverage holes
- Refine and enhance the coverage model

cadence™

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

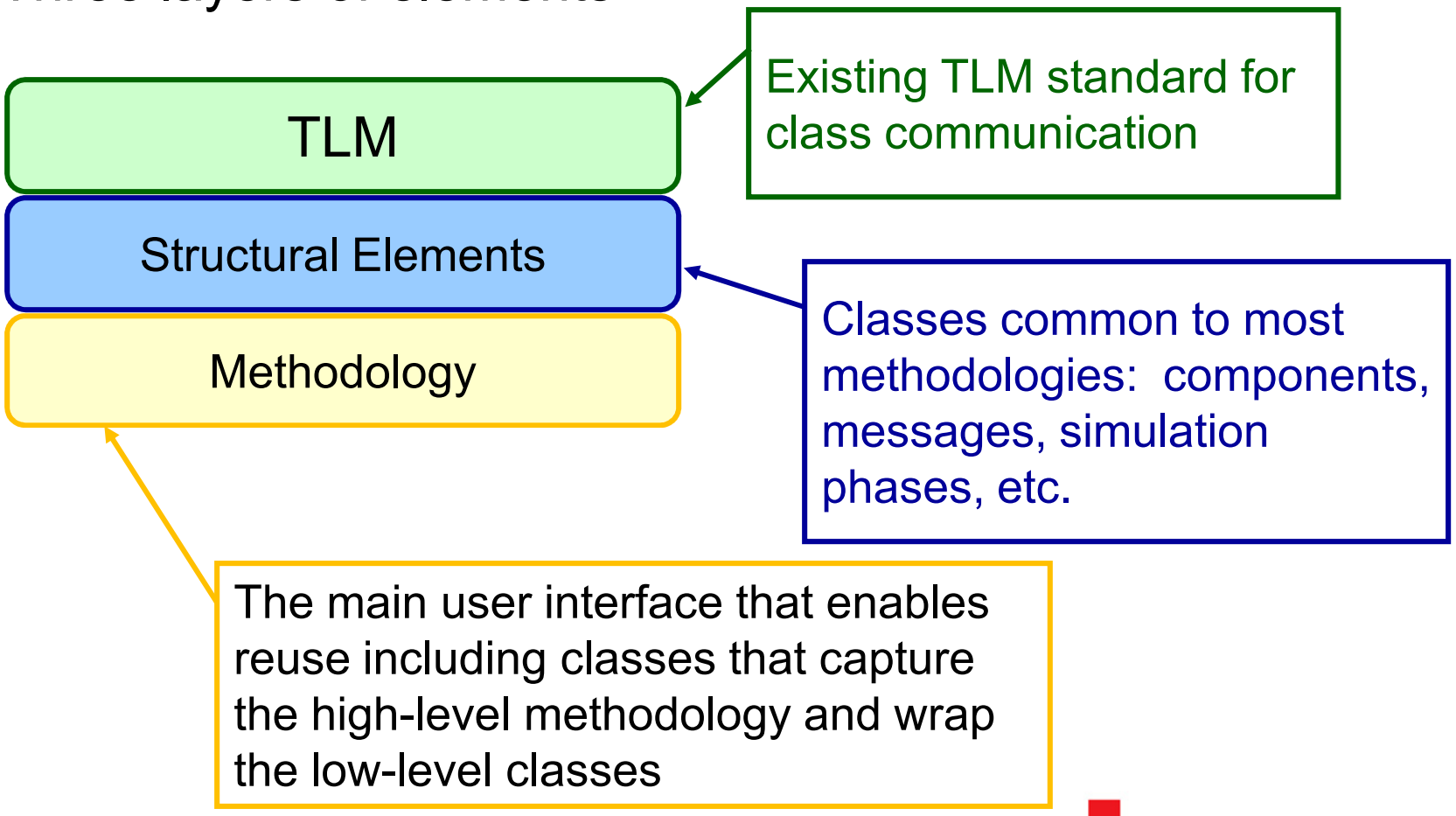
Summary

cadence™

The UVM Library

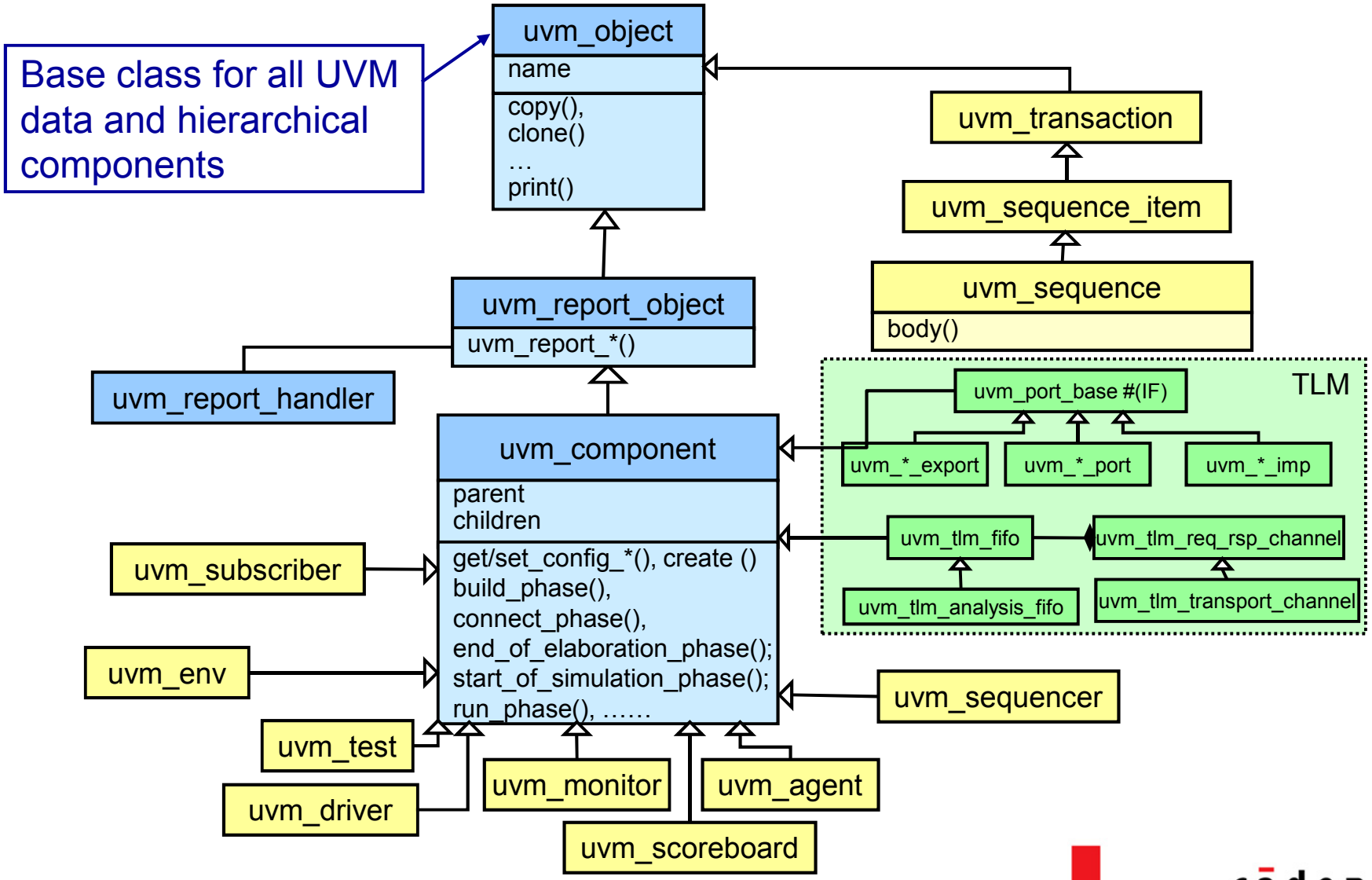
Using inheritance and encapsulation in class libraries

- Three layers of elements



cadence

UVM Class Hierarchy
(UVM 1.1)



cadence

Using the UVM Class Library

Deriving an environment from the class library allows:

- Exploiting built-in automation
  - Sequences … standard & powerful test writer interface
  - Built-in factory for rapid reuse
  - Transaction manipulation (packing, deep copying etc)
  - Flexible automatic test phase interface
  - Messaging utilities
- Adhering to a proven methodology
  - Modular topology (monitors, drivers, agents)
  - Out-of-the box or partial reuse
  - TLM communication for language independent modularity
  - A powerful component configuration scheme

cadence™

Where Do I Get UVM?

Option 1

Download the UVM library from
www.uvmworld.org

You will be redirected to
http://www.accellera.org/activities/vip

Option 2

Included in Incisive Release

- Located at: `` `ncroot`/tools/uvm ``
- Will contain bug fixes not on UVM world
- Contains Cadence additions to UVM
  - Tcl commands
  - Transaction recording
  - Register generator
  - Templates for verification component generation

cadence™

What the release looks like?

`uvm/`	
`  UVM_Reference.html`	UVM Reference Manual
`  uvm_release-notes.txt`	UVM Release information
`uvm/bin/`	
`  ovm2uvm.pl`	OVM to UVM migration script
`uvm/doc/`	User Guide and Reference
`uvm/src/`	UVM source files
`  base/`	low-level library components
`  comps/`	methodology layer (agent, env)
`  reg/`	register package
`  macros/`	macro definitions
`  seq/`	sequence-related code
`  tlm1, tlm2/`	TLM layer components
`  uvm_pkg.sv`	`include file required for UVM
`  uvm.sv`	
`  uvm_macros.svh`	

cadence™

What the release looks like? (cont.)

`uvm/sv/cdns_additions`	Cadence-specific additions (only found in the IUS version of UVM)
`uvm/examples/`	Directories of examples that run on multiple simulators
`simple/`	Includes: basic_examples, callbacks,
.....	configuration, factory, phases, etc
`integ/`	more complex UVM examples
`xbus/`	UVM equivalent of the xbus
`examples/`	To run on IES: irun –f compile_ius.f
`sv/`	
`codec/`	
`apb/`	

cadence™

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
  - Modeling Data Items
  - Reusable Sequences
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

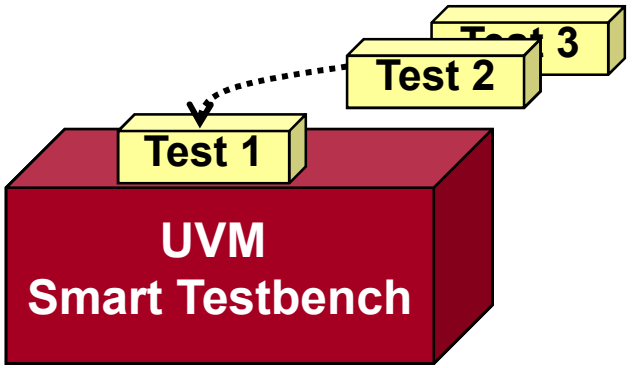
UVM Multi-Language and UVM Acceleration

Summary

cadence™

Modeling Data Items

- Data items
  - Represent the main transaction input to the DUT
  - Adhering to a protocol, consistent values are generated and sent
  - Examples include packets, transactions, instructions, and so on
- Test environment randomizes data items (transactions)
  - A default distribution should be generated
  - Tests further steer generation by layering constraints or selecting from pre-defined scenarios
  - This makes test short, readable, easier to write and maintain



cadence

SystemVerilog Data Item: A UART Frame

```
class uart_frame;
  rand int transmit_delay;
  rand bit start_bit;
  rand bit [7:0] payload;
  rand bit [1:0] stop_bits;
  rand bit [3:0] error_bits;
  bit  parity;

  // utility functions
  extern function bit calc_parity( );
  extern function void print( );
  extern function bit compare(uart_frame rhs);
  …
endclass: uart_frame
```

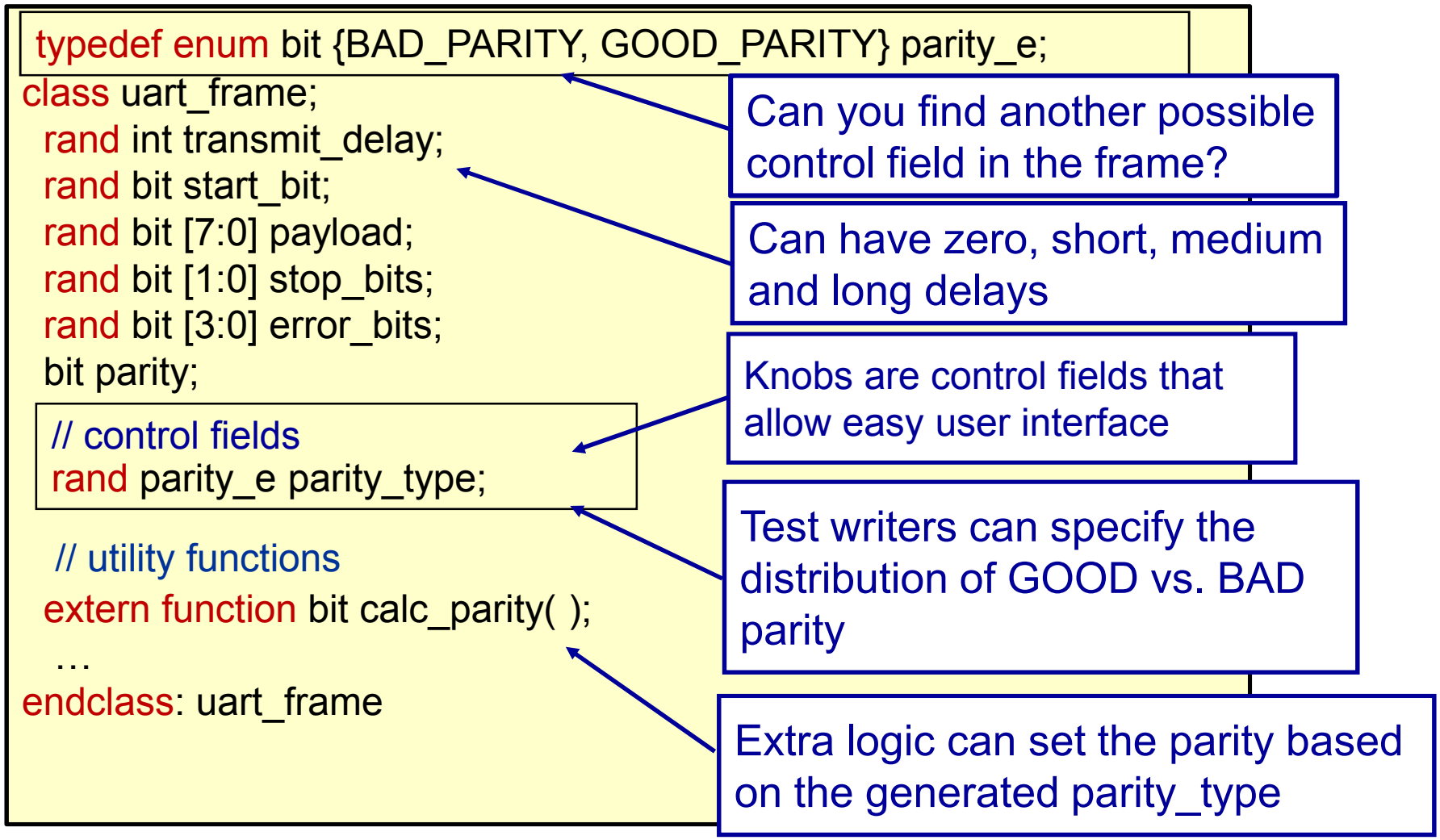
Fields to be randomized

How will the test-writer request a bad parity frame?

parity is calculated using a function

cadence™

Design for Test-writing using Knobs



Recommendation: Think of the desired scenarios and make sure you have the needed control knobs to achieve them

cadence™

Data Item Example

```
class uart_frame;
  rand int transmit_delay;
  rand bit start_bit;
  rand bit [7:0] payload;
  bit parity;
  rand bit [1:0] stop_bits;
  rand bit [3:0] error_bits;

  // Utility functions
  extern function bit calc_parity( );

  extern function void print();
  extern function bit compare(uart_frame rhs);
  …
endclass: uart_frame
```

User defined function

Required for most transactions!

cadence™

Where SV Language Stops and UVM Begins

Example: Data Items

```
class uart_frame;
   rand  bit         start_bit;
   rand  bit [7:0]   payload;
   rand   bit        parity;
   rand  parity_e    parity_type;
   rand  int         transmit_delay;
endclass: uart_frame
```

Does language alone support all the necessary customization operations?

- Randomization

No! Only randomization is defined in the SystemVerilog LRM

- Printing
- Cloning
- Comparing
- Copying
- Packing
- Transaction Recording

UVM provides the rest!

cadence™

Enabling Data Item Automation

```
class uart_frame extends uvm_sequence_item;
  rand  bit         start_bit;
  rand  bit [7:0]   payload;
  bit               parity;
  rand  parity_e    parity_type;
  rand  int         xmit_delay;

  // *optional* field declarations and automation flags
  `uvm_object_utils_begin(uart_frame)
    `uvm_field_int( start_bit,  UVM_ALL_ON)
    `uvm_field_int( payload,  UVM_ALL_ON)
    `uvm_field_int( parity,  UVM_ALL_ON)
    `uvm_field_enum( parity_e, parity_type, UVM_ALL_ON + UVM_NOCOMPARE)
    `uvm_field_int( xmit_delay,  UVM_ALL_ON + UVM_DEC + UVM_NOCOMPARE)
  `uvm_object_utils_end
endclass: uart_frame
```

UVM data items are derived from **uvm_sequence_item**

Enables all automation for **uart_frame** fields

Specify field level flags: **UVM_NOCOMPARE, UVM_NOPRINT**, etc.

cadence™

Example: Data Type Automation

```
uart_frame frame1, frame2;
initial
begin
   frame1 = new("my_frame");
   assert(frame1.randomize());
   frame1.print();
   $cast(frame2, frame1.clone());
   frame2.set_name("my_frame2")
   frame2.print(tree_printer);
 end
```

```
-------------------------------------
Name            Type      Size   Value
-------------------------------------
my_frame        uart_frame -    @607
  start_bit     integral   1    'h1
  payload       integral   8    'hfc
  parity        integral   1    'h0
  parity_type   parity_e   32   BAD_PARITY
  xmit_delay    integral   32   'd382
-------------------------------------
```

Create a new uart_frame and copy frame1 to frame2

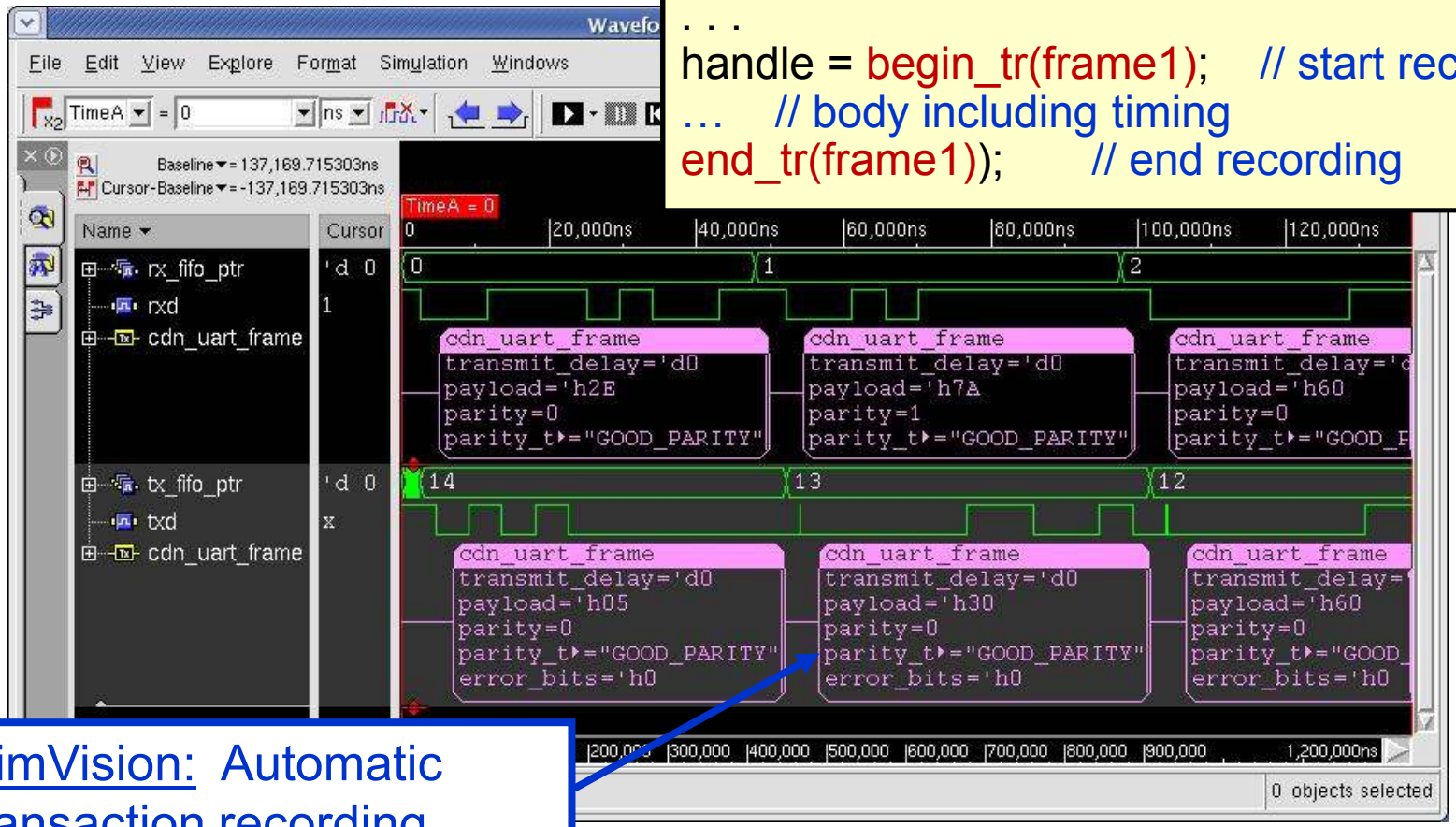
```
  my_frame2:  (uart_frame@788) {
    start_bit: 'h1
    payload: 'hfc
    parity: 'h0
    parity_type: BAD_PARITY
    xmit_delay: 'd382
}
```

Three built-in printers can be used

cadence™

Transaction Recording in IES

```
// inside driver or monitor code
uart_frame frame1;
int handle;
. . .
handle = begin_tr(frame1);    // start recording
…    // body including timing
end_tr(frame1));      // end recording
```



SimVision: Automatic transaction recording.

Automatic recording methods built into UVM items

cadence

Messages for Debug & Error Reporting

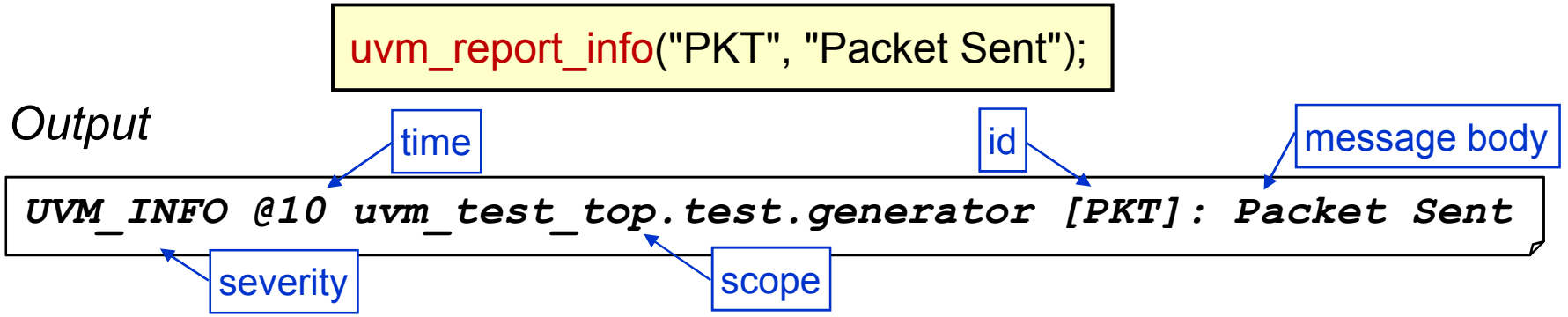
- Messages
  - While developing or using environments, users need to print messages
  - $display does not allow non-intrusive filtering and control
  - User may want to get trace messages from a suspect component
  - Critical to large environment with many verification components
- Error reporting
  - The testbench must produce clear and descriptive error messages to communicate a checker mismatch
  - Should affect simulation based on severity
    - Error – stop
    - Warning – record and proceed
    - Allow customization of above

cadence™

UVM Messaging Facility

Messages print trace information with advantages over $display:

- Aware of its hierarchy/scope in testbench
- Allows filtering based on **hierarchy**, **verbosity**, and **time**



- Simple Messaging:
  - uvm_report_*(string id, string message);
    - Where * (severity) is one of fatal, error, warning, info
- Tips:
  - Use get_type_name() to give an id of the class type
  - Use $sformatf() to create message string using $display syntax

cadence

Messages and Verbosity

```
uvm_report_info(get_type_name(), $sformatf("addr %h",addr), UVM_LOW);
```

- Additional argument to uvm_report* defines verbosity
  UVM_NONE, UVM_LOW, UVM_MEDIUM, UVM_HIGH, UVM_FULL

- By changing the verbosity maximum, messages can be filtered
  - For example, if the verbosity is set to UVM_LOW, all messages coded with UVM_MEDIUM and higher will not print

Ways to change verbosity *without recompiling!*

1. Command-line: % irun …. +UVM_VERBOSITY=UVM_NONE
2. Procedural (test): set_report_verbosity_level_hier(UVM_MEDIUM);

cadence™

Using the UVM Library with Incisive Simulators

- **irun** allows users to run the simulator by specifying all input files and command-line options on a single command line.
  - Each file is compiled using the appropriate compiler based on the file extension used.
    - Verilog, SystemVerilog, VHDL, *e*, C and C++
  - Command-line options are also recognized and passed to the compiler, the elaborator and the simulator

```
% irun -access +r -gui test.sv  verify.e  top.v  sub.vhd
```

- **irun** supports UVM-specific options:

-uvm	Searches *<install_dir>/tools/uvm/src* and sets command-line options to compile the lib
-uvmhome $UVM_HOME	Optionally specifies a different directory for the UVM installation
+UVM_TESTNAME	Passes the default test name for run_test()
+UVM_VERBOSITY	Sets the verbosity level for UVM messages

cadence™

Constraint Layering Using the Factory

```
typedef enum {SHORT, MEDIUM, LONG} delay_t;
// extended class uses control knob for transmit_delay
class delay_frame extends uart_frame;
  rand delay_t delay_knob;
  constraint delay_type_c {      transmit_delay > 0;
              delay_knob == SHORT -> transmit_delay < 10;
              delay_knob == MEDIUM -> transmit_delay < 40
              delay_knob == LONG -> transmit_delay > 100;  }
  `uvm_object_utils_begin(delay_frame)
     `uvm_field_enum(delay_t, delay_knob, UVM_ALL_ON|UVM_NOCOMPARE)
  `uvm_object_utils_end

  function new (string name="delay_frame"); …
endclass: delay_frame
```

Add control knob to constrain transmit_delay

Registers delay_frame with the factory

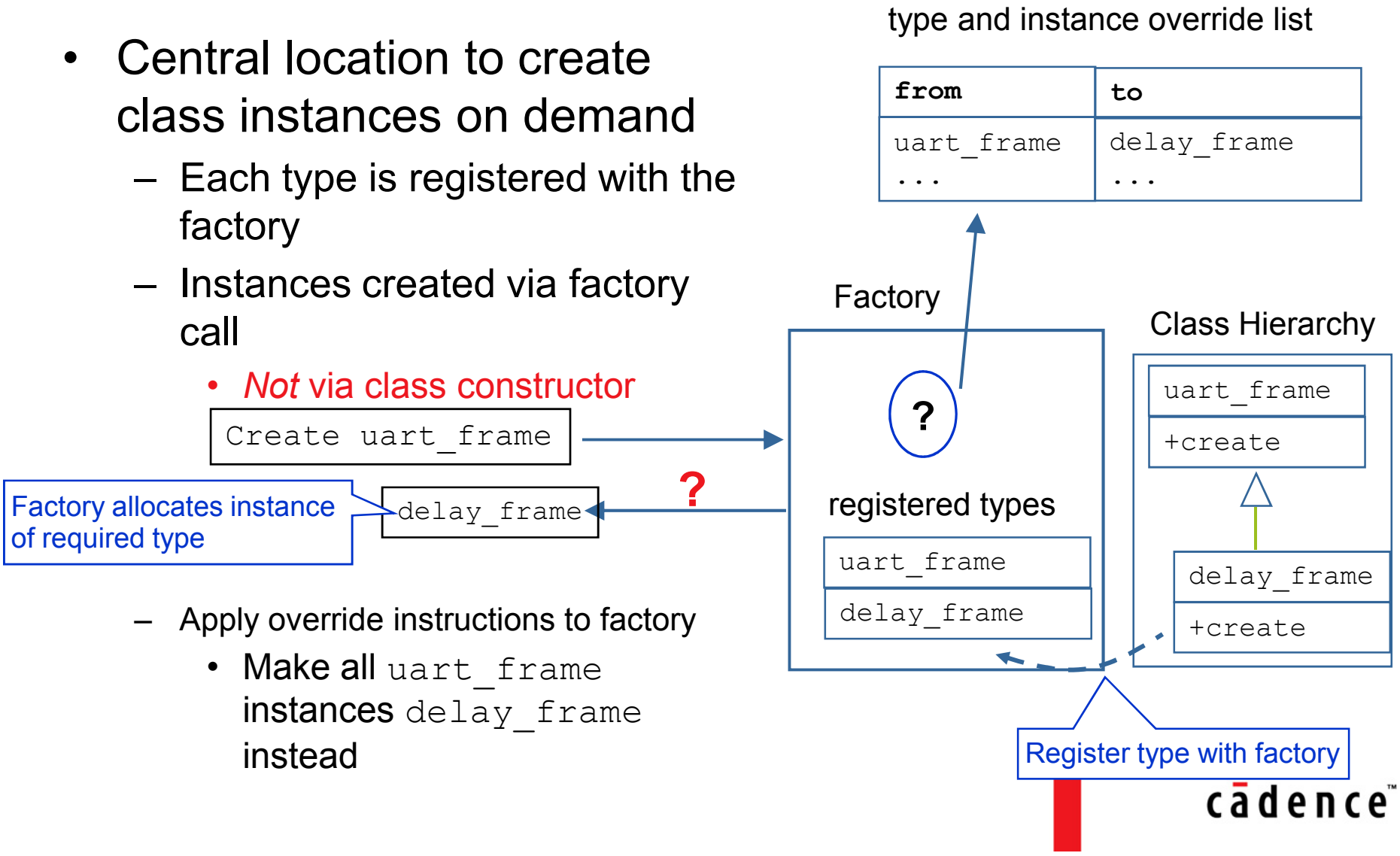
```
// In the generator class:
uart_frame  frame;    // data item declaration
. . .
frame = uart_frame::type_id::create("frame"));
```

Replace frame=new(); with create (…)

cadence™

What is the UVM Factory?

- Central location to create class instances on demand
  - Each type is registered with the factory
  - Instances created via factory call
    - *Not* via class constructor



  - Apply override instructions to factory
    - Make all `uart_frame` instances `delay_frame` instead

cadence

Factory Overrides

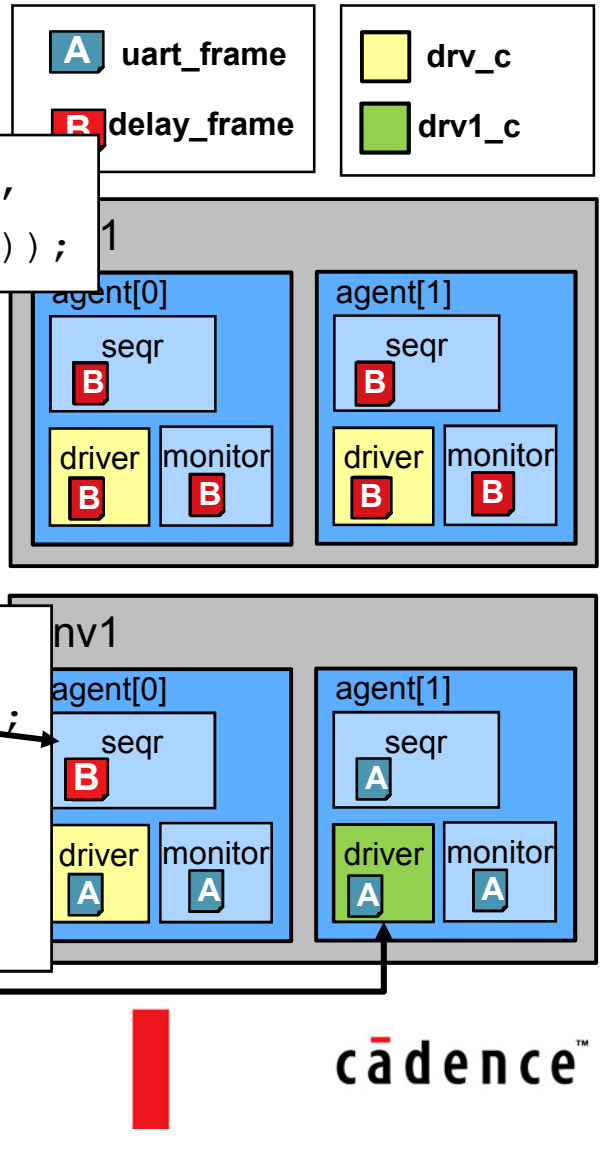
- Type override replaces ALL instances:-
  - For a specific test, replace all `uart_frame` packets with `delay_frame` packets

```
set_type_override_by_type(uart_frame::get_type(),
                          delay_frame::get_type());
```

- Instance override replaces specific instances:
  - Replace `uart_frame` in agent[0] sequencer
  - Replace `agent[1]` driver `drv_c` with `dvr1_c`

```
set_inst_override_by_type("env1.agent[0].seqr",
  uart_frame::get_type(),delay_frame::get_type());

set_inst_override_by_type("env1.agent[1]",
  drv_c::get_type(), drv1_c::get_type());
```



cadence

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
  - Modeling Data Items
  - Reusable Sequences
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

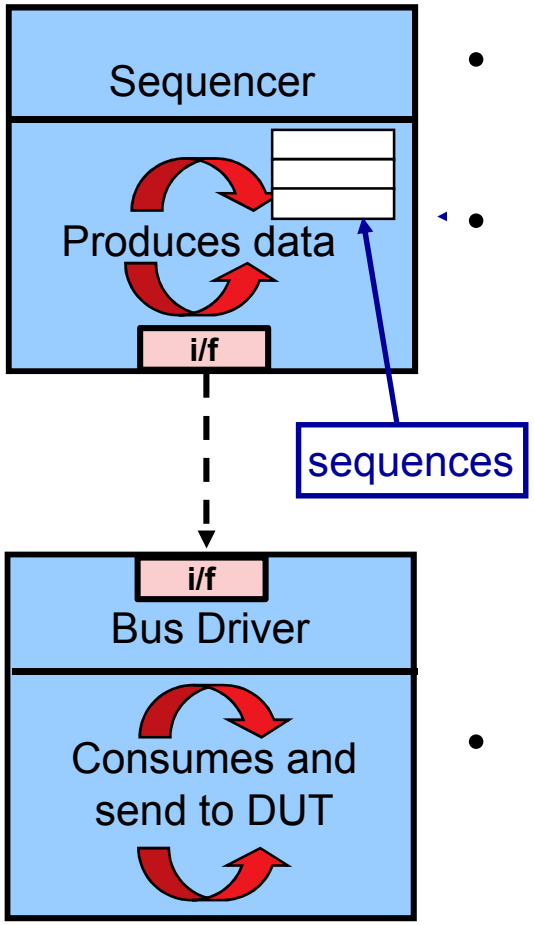
cadence™

Scenario Creation Requirements

- Achieving full coverage using constraint layering is highly recommended
  - The technology evenly selects items from legal ranges
  - Efficient in capturing unanticipated bugs
- Many times, single items cannot capture high-level intention
  - Ordered streams of transactions are required
  - Examples: configuration of a device, program generation
- A sequence is a set of transactions that accomplish a defined task
- Sequences are provided as part of a reusable component
  - Capture important scenarios that should be exercised
- UVM provides an advanced sequence generation mechanism

cadence™

UVM Sequencer – an Advanced Generator



UVM sequencer

- By default works exactly like a generator
  - Generates random transactions on request
- Addresses all randomization requirements:
  - random by-default, reactive generation, system-level ready, modeling timing, etc
- User-defined sequences of transactions are created and executed
  - For example, a user creates a sequence that interleaves legal and illegal frames
- Test writer sets the sequencer to execute a specific sequence

cadence™

Sequences Example: UART Retry Sequence

```
// Send one BAD_PARITY frame followed by a GOOD_PARITY
//  frame with the same payload
class retry_seq extends uvm_sequence #(uart_frame);
  // uart_frame req;    // built-in data item field
  `uvm_object_utils(retry_seq)

  rand bit [7:0] pload;    // randomizable sequence parameter
  . . .
  virtual task body ( );   // sequence behavior
    `uvm_do_with(req, {payload == pload; parity == BAD_PARITY;} )
    `uvm_do_with(req, {payload == pload; parity==GOOD_PARITY;} )
  endtask : body
endclass
```

extend from uvm_sequence

Data item

Registers the retry_seq with the factory

body() can also include time consuming statements, fork…join, function calls, etc

cadence™

The ‘uvm_do’ Operation

```
…
virtual task body ();   // sequence behavior
    `uvm_do_with(req,  {payload == pload; parity == BAD_PARITY;} )
endtask
```

Reactive generation

- Translates into:
  - Wait till item is requested
  - Create items using a factory
  - Randomize (with constraints or not)
  - Return the item to the requester
  - Block code execution till item_done()

• Combines procedural and declarative (constraint layering) directives
• Factory support out-of-the-box

Inline constraints for additional control

• Allows modeling timing
• e.g. scenario that calls for

• built-in TLM to allow reuse and support multi-language

New and expert users deploy all key randomization concepts!

cadence™

Nesting Sequences

```
// call retry sequence wrapped with random frames
class rand_retry_seq extends uvm_sequence #(uart_frame);
  `uvm_object_utils(rand_retry_seq)

  retry_seq retry_sequence;

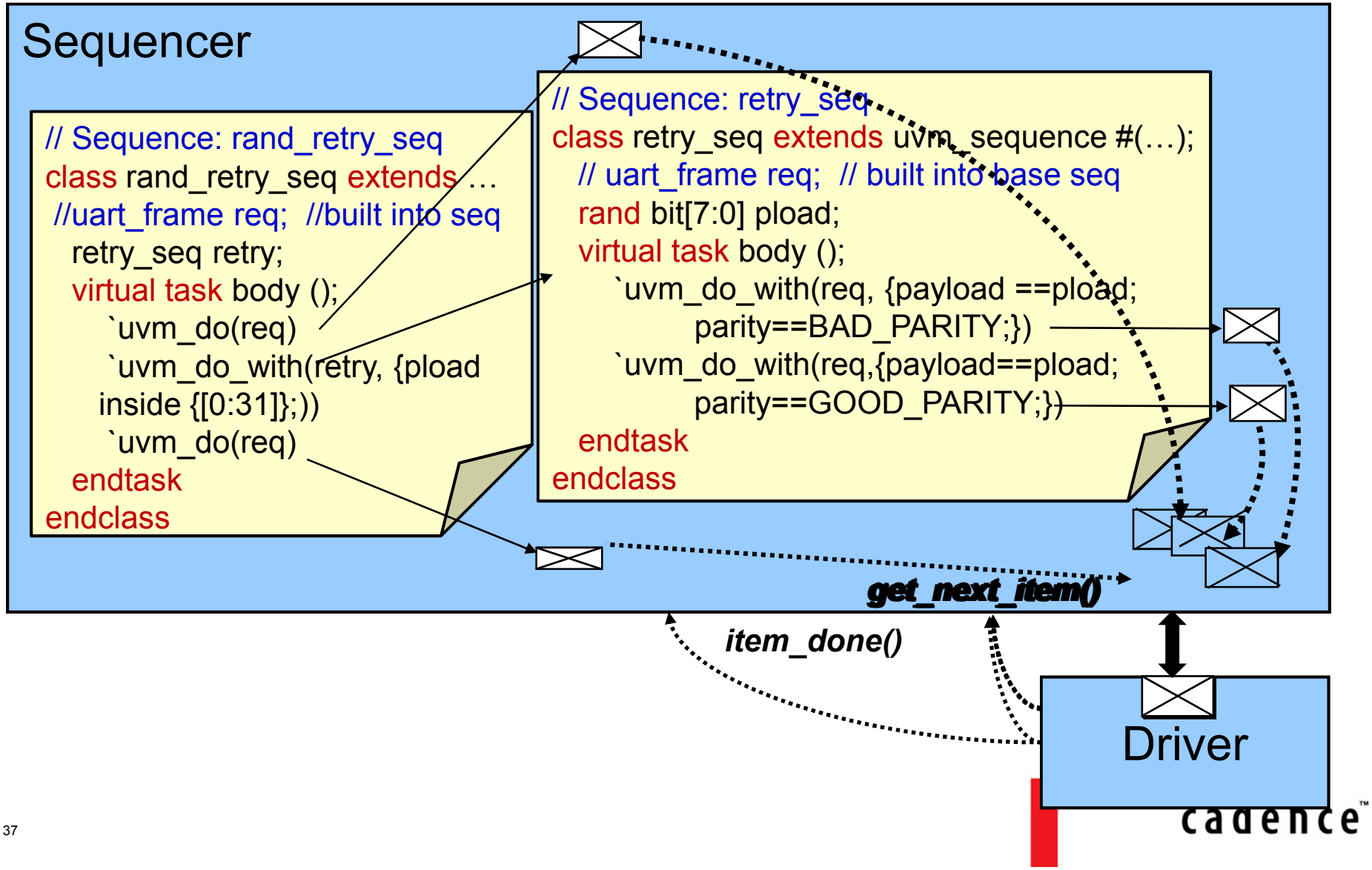
  virtual task body (); // executable sequence behavior
     `uvm_do (req)
     `uvm_do_with(retry_sequence , {pload inside {[0:31]};})
     `uvm_do(req)
  endtask
endclass
```

Using a previously defined sequence

Create and randomize a retry_seq sequence and call it's body

cadence™

Sequence Execution



cadence™

Controlling Sequencer Behavior

- How do we control which sequences run on a UVC sequencer and when?
  1. `run_phase()` (and every run sub-phase) has a `default_sequence` property.
     - Set `default_sequence` to a sequence to execute in that phase.

```
// Test: Set uart_tx_sequencer  to use retry_seq as the default sequence
uvm_config_db#(uvm_config_wrapper)::set(this,"uart_tx_sequencer.run_phase
    ",    "default_sequence",
          retry_seq::type_id::get());
```

  2. Execute a sequence directly on a UVC sequencer from a test class, by calling the sequence's start method

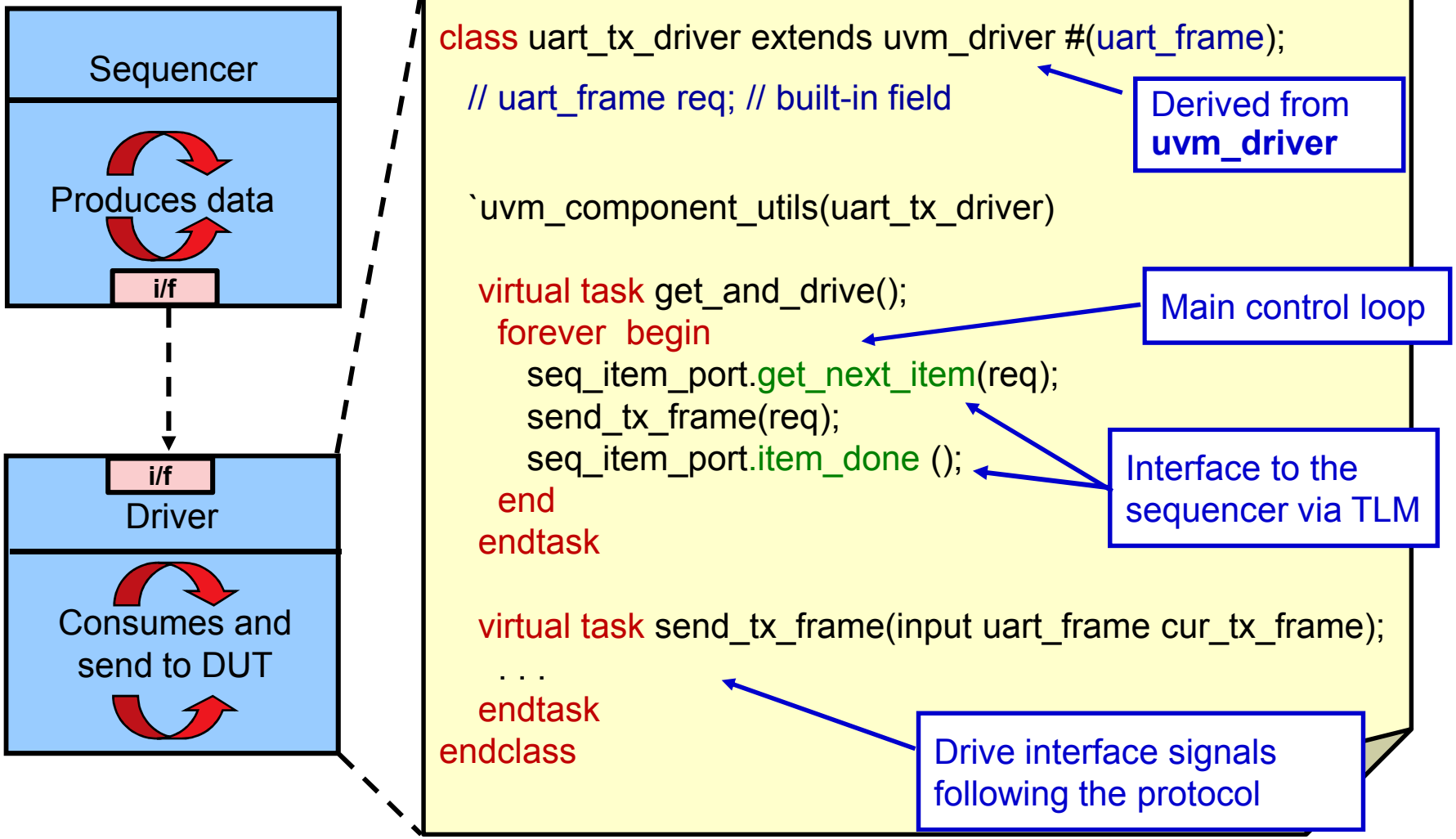
  3. Group sequences into a sequence library; Set `default_sequence` to execute the sequence library

Notes: 2 & 3 will be covered in UVM Training class

cadence™

The UVM Driver

Pulls transactions from the sequencer and drives the DUT interface



cadence

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

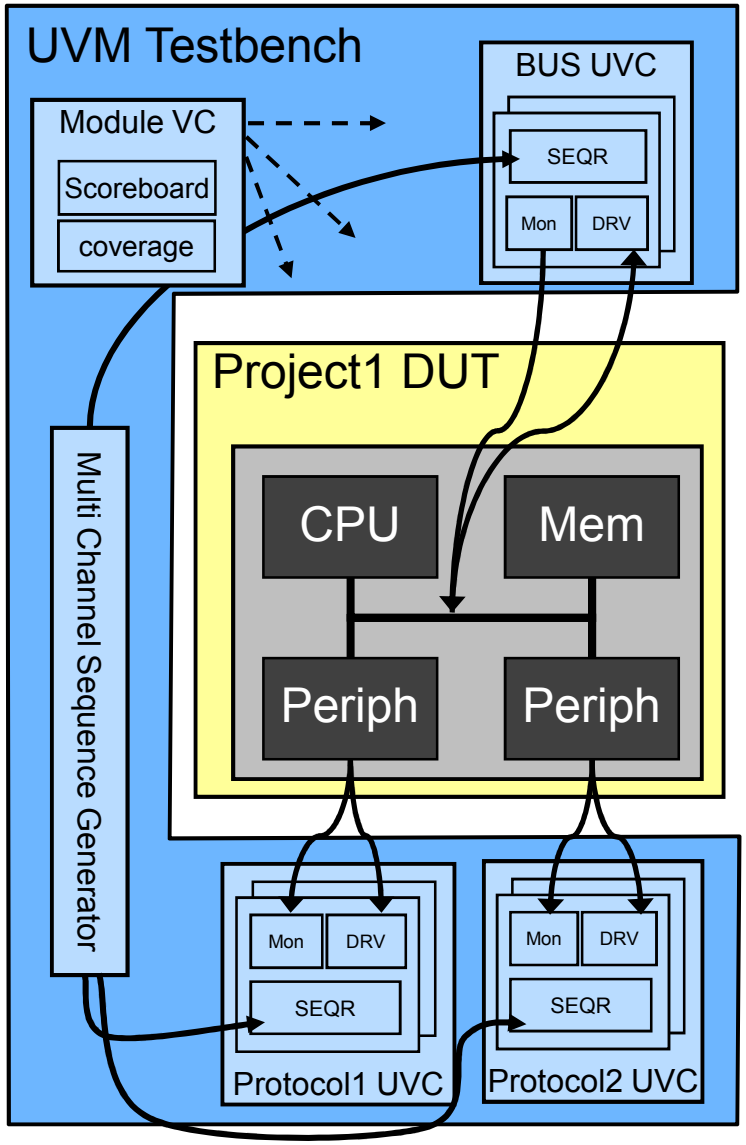
- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
  - Concepts for reuse
  - Reusable environment topology
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

cadence™

Creating a Reusable Verification Environment

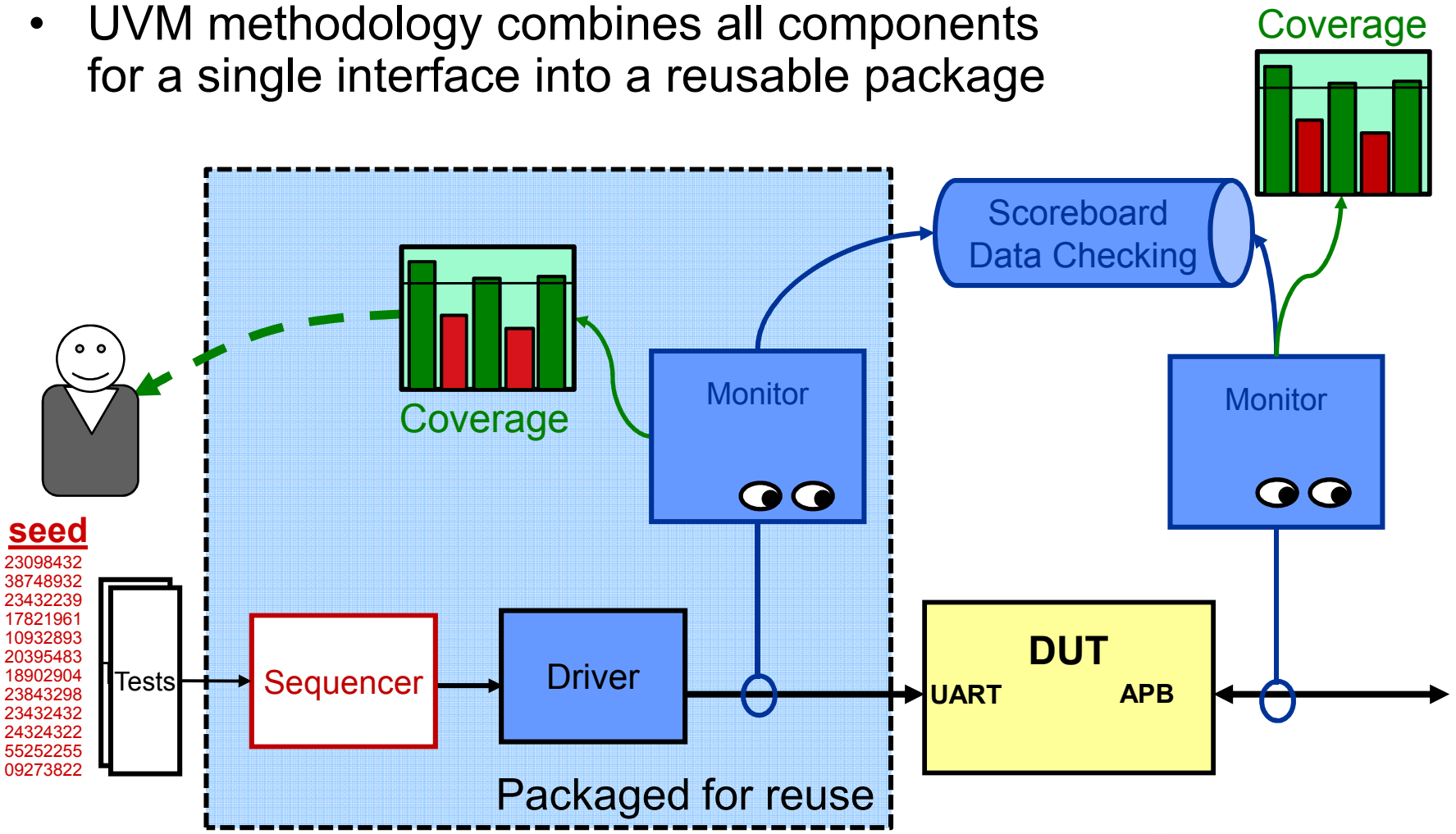


- Tasks to be done:
  - Apply stimulus on each external interface
  - Coordinate stimulus across multiple ports for system-level scenarios
  - Emulate the CPU by reading and writing device registers
- Issues:
  - How do we model complex environments? Generation? Checking? Coverage?
  - How do we facilitate reuse?
- Solution: “divide and conquer”
  - Create a reusable verification component per interface
  - Following the protocol, each component:
    - Stimulates the interface
    - Checks for protocol correctness
    - Collects coverage
  - Add a mechanism for multi-channel traffic generation
  - Create end-to-end checker and coverage collector

cadence™

Packaging UVM Components for Reuse

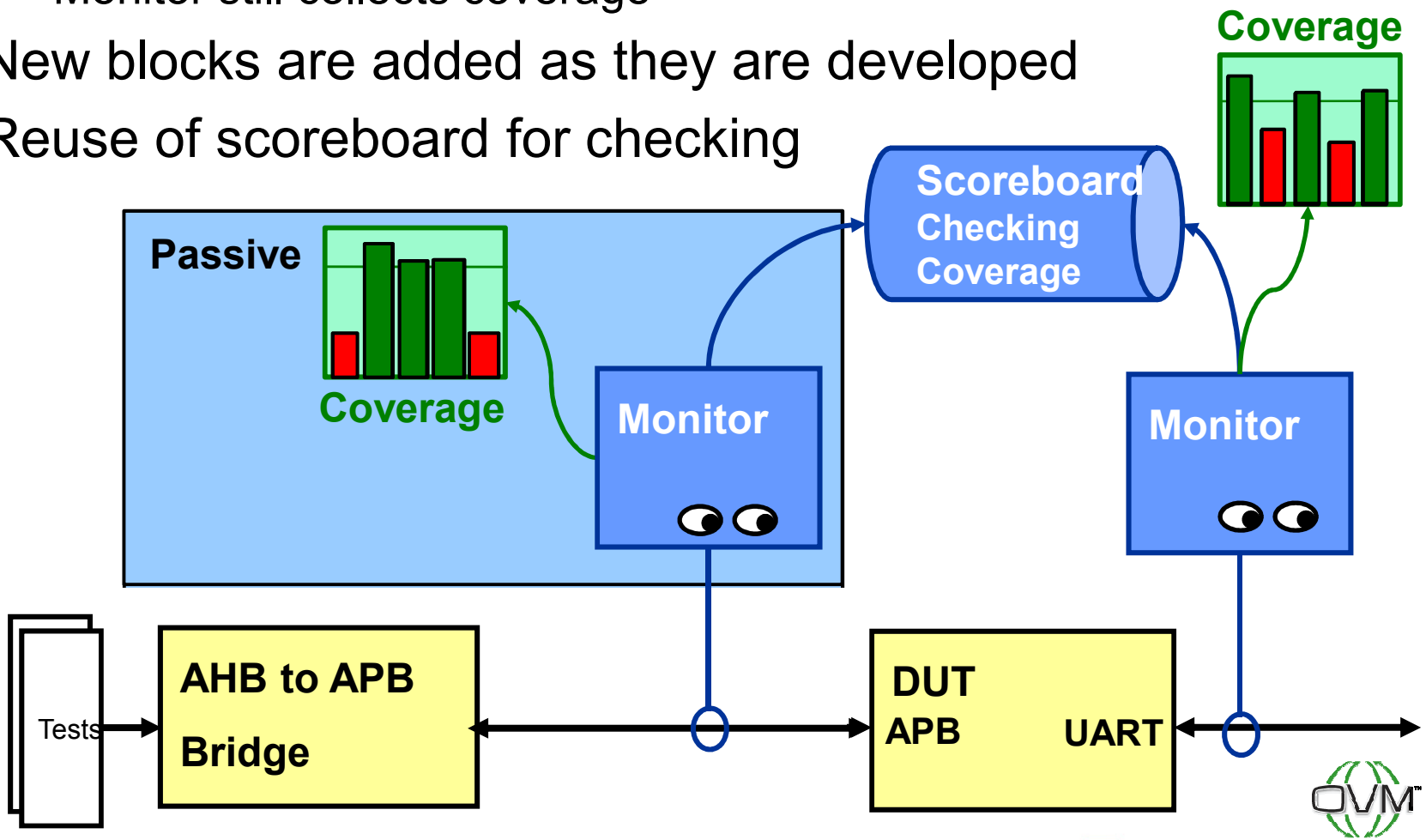
- UVM methodology combines all components for a single interface into a reusable package



cadence

Reuse at the System Level

- Configure APB UVC as passive:
  - Monitor still collects coverage
- New blocks are added as they are developed
- Reuse of scoreboard for checking



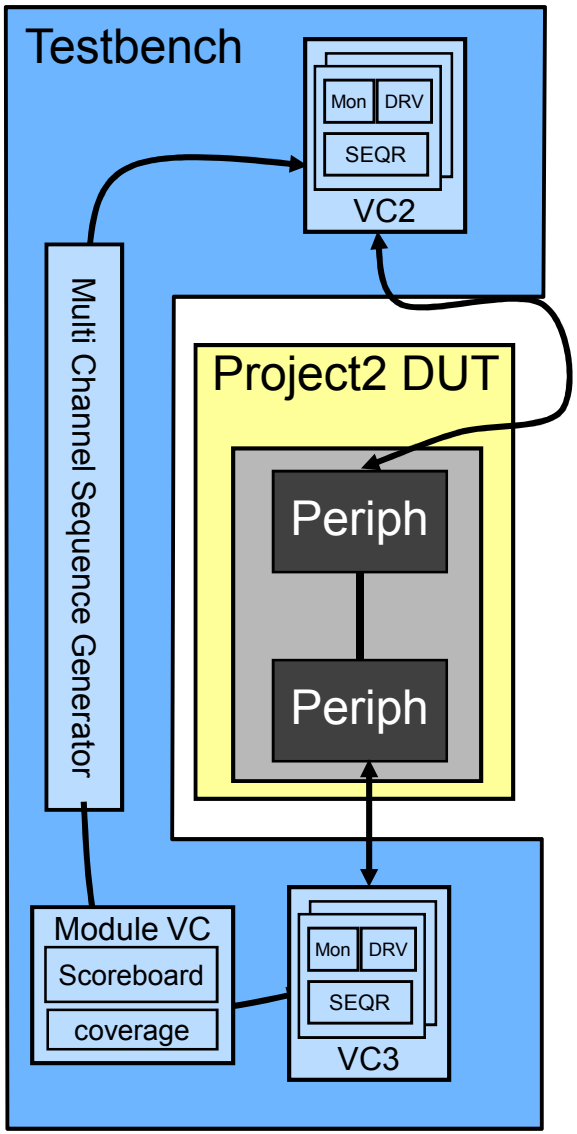
Reusable Environments: Main Players

- Contributors have different skill sets and responsibilities
- Who are they?
  - Reusable component developers
    - Experts in protocol knowledge and verification
  - Environment integrators
    - Build test environments using reusable components
    - Less protocol knowledge, more target system knowledge
  - Test-writers
    - May have little verification expertise
- UVM allows each contributor to focus on his responsibilities and be productive quickly

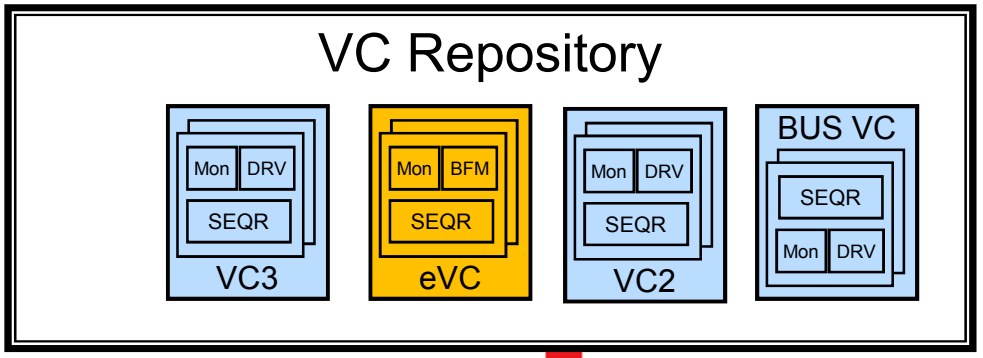
Note: At times the same person is doing two or all the roles above

cadence™

Facilitating Reuse

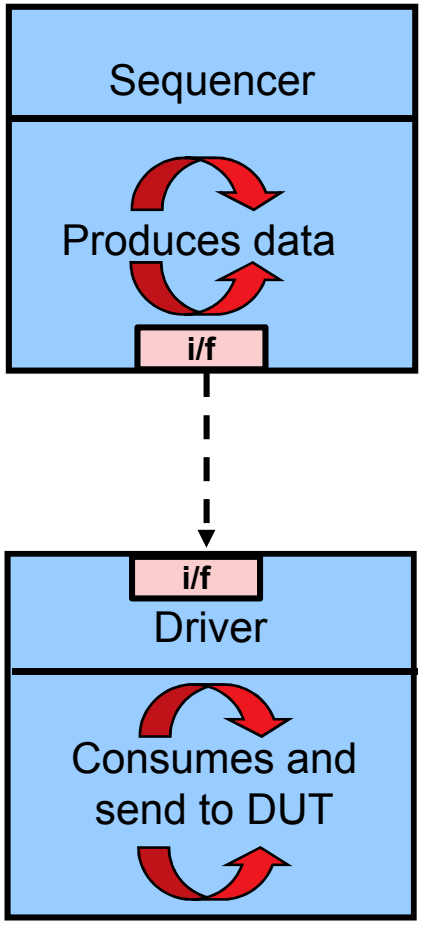


- Create a components repository
- Reuse existing components for next project
  - Can use various languages based on needs and preferences
- New components are added as needed
- Over time, components repository grows and matures
  - Proven on multiple projects
  - Becomes a major company strength!



cadence

Coverage and Checking

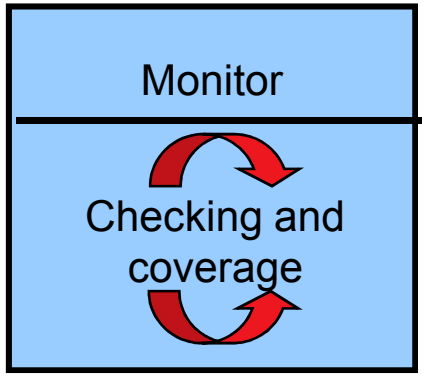


- From where should we collect coverage?
- Neither the sequencer nor the driver are the ideal option
  - What if we want to monitor an interface that we do not drive?
- For this we need a third component called a “Monitor”

cadence™

The Monitor

- Monitor
  - A passive component that collects information on a monitored interface
  - Builds abstract transactions
  - Contains events, status, checkers and coverage
- Protocol specific
- Passive entity
  - Never drives signals!
- Monitor is independent of the driver!
  - May need to monitor information driven by the testbench



cadence™

The UVM Monitor – Coverage Implementation

```
class uart_monitor extends uvm_monitor;
  `uvm_component_utils(uart_monitor)
   uart_frame  frame;     // data item to be collected

  covergroup uart_trans_frame_cg;
    NUM_STOP_BITS : coverpoint  frame.nbstop {
                bins ONE = {0};
                bins TWO = {1};
                }
    DATA_LENGTH : coverpoint  frame.char_length { …}
    PARITY_MODE : coverpoint  frame.parity_mode { … }
  endgroup
   function new (string name = "", uvm_component parent = null);
       super.new(name, parent);
       uart_trans_frame_cg = new();
   endfunction: new
   task collect_frame ( );
     // Collect uart data from interface
     uart_trans_frame_cg.sample();
   endtask: collect_frame
endclass: uart_monitor
```

Derived from uvm_monitor and registered with the factory

Implement coverage model based on your verification plan

new() covergroup in the constructor

Task to collect data items from the DUT interface

Sample coverage when frame is collected

cadence™

Creating Assertion/Formal-Ready Interface Components

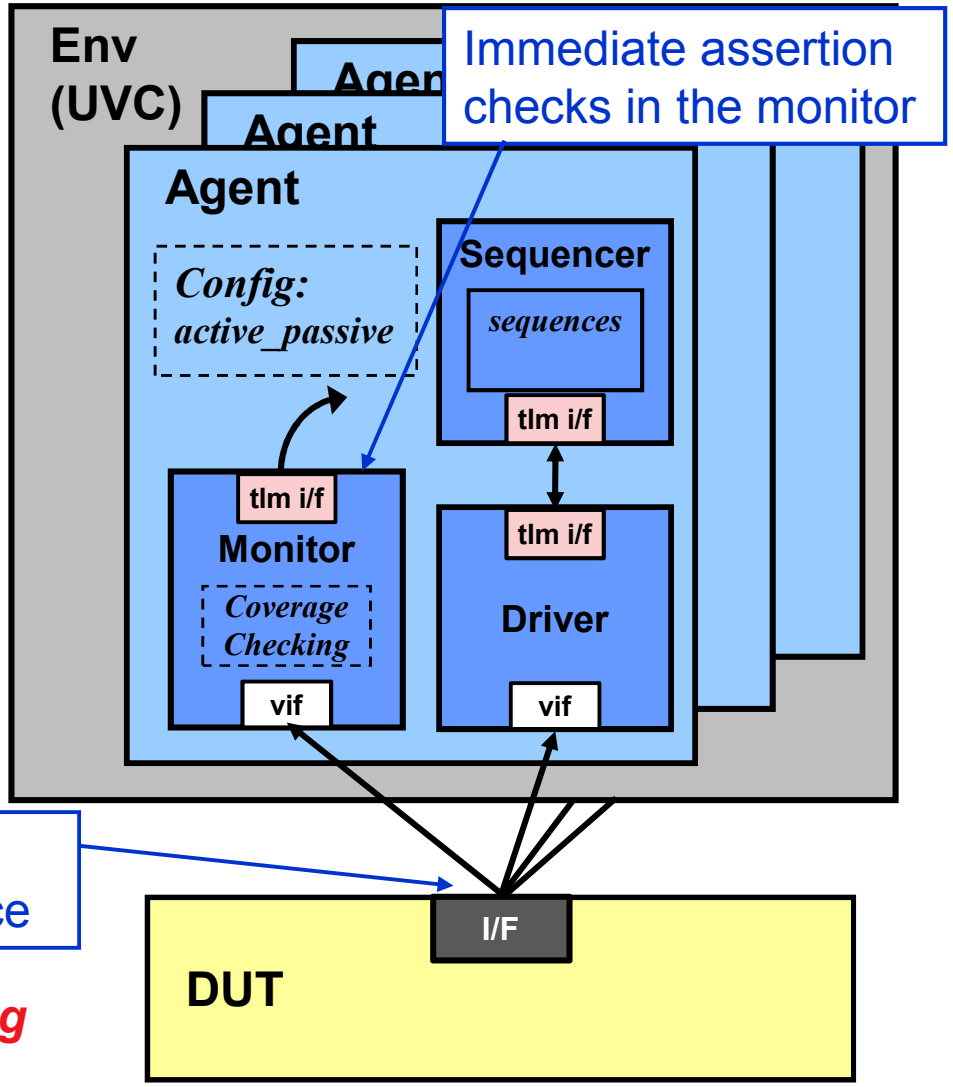
- Assertions are useful for dynamic simulation
  - Can identify overlooked issues
    - DUT and reference model can be wrong
  - Can significantly reduce debug time vs. end-to-end checkers
- Can be leveraged in formal verification
  - Another efficient, complementary path to discover bugs
  - Can be exploited before a dynamic environment exists
- Interface UVCs are protocol specific
  - Ideal place to capture protocol assertions inside
- UVM introduces a unified flow for dynamic and formal verification

cadence™

UVM Assertions Methodology

The methodology was designed to:

- Combine upfront planning and assertions development into your flow
- Instantiate and control assertions
- Reuse assertions
  - Between projects
  - Across teams (designers, verification engineers, etc)
  - Using multiple technologies
- Measure thoroughness using assertions



Ask us for information or T=training on UVM and Assertions

cadence

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

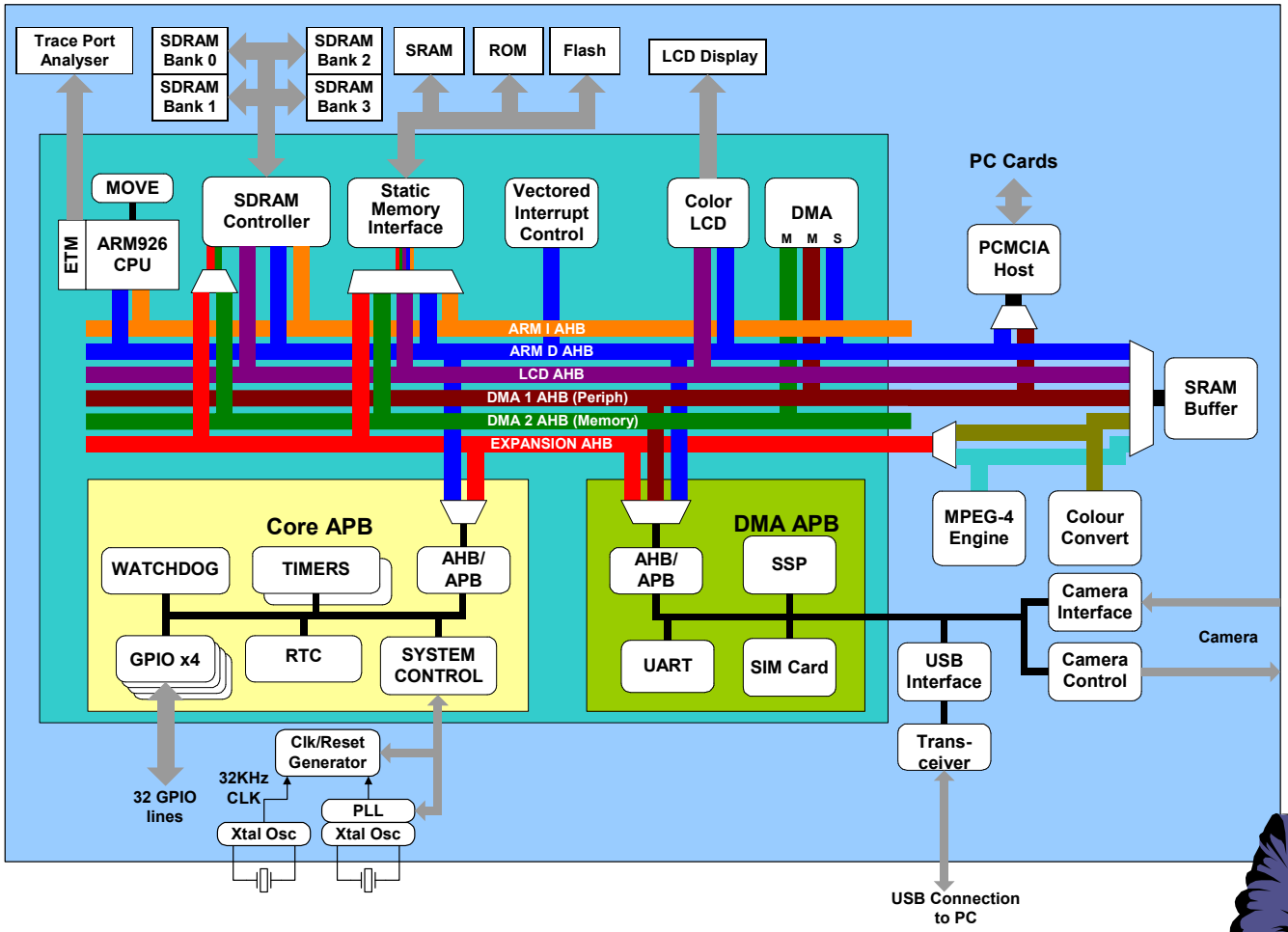
- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
  - Concepts for reuse
  - Reusable environment topology
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

cadence™

Consider This: Your Next Verification Challenge!

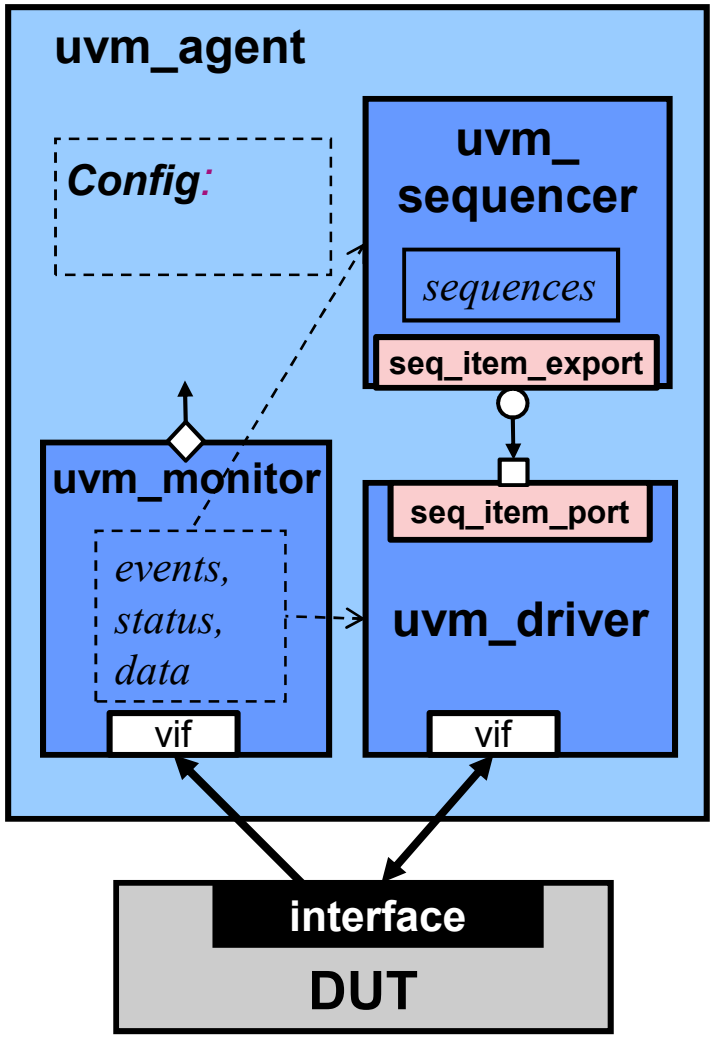




Scalability is a must! Class level reuse is not enough!

cadence™

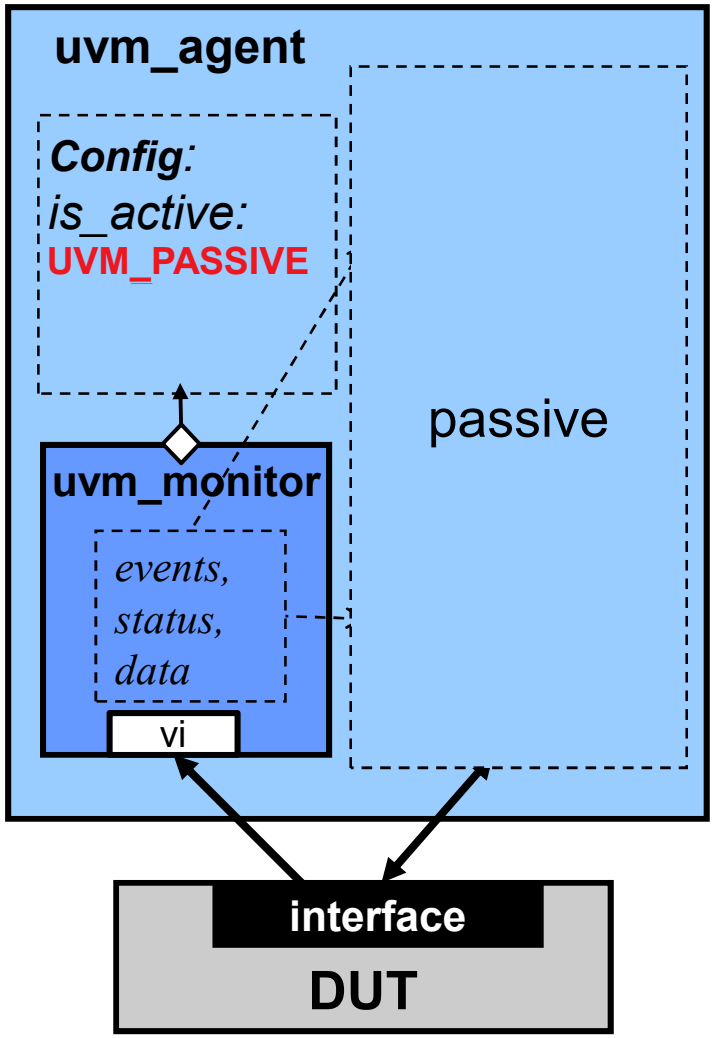
The Solution: Agents



- Agents provide all the verification logic for a device in the system
- Instantiation and connection logic is done by the developer in a standard manner
  - Integrator does not need to worry about this
- A standard agent has:
  - Sequencer for generating traffic
  - Driver to drive the DUT
  - Monitor
- Note: that the monitor is independent of the driving logic to allow a passive operation mode
- Agent has standard configuration parameters for the integrator to use

cadence

Agent Standard Configuration



- A standard agent is configured using an enumeration field: **is_active**

  UVM_ACTIVE:
  - Actively drive an interface or device
  - Driver, Sequencer and Monitor are allocated

  UVM_PASSIVE:
  - Only the Monitor is allocated
  - Still able to do checking and collect coverage

cadence

Creating the Agent

```
class master_agent extends uvm_agent;
    uvm_active_passive_enum  is_active = UVM_ACTIVE;

   `uvm_component_utils_begin(master_agent)
     `uvm_field_enum(uvm_active_passive_enum, is_active, UVM_ALL_ON)
   `uvm_component_utils_end
   // agent sub component instances
    master_driver          driver;
    master_sequencer    sequencer;
    master_monitor         monitor;
   function new( input string name, input uvm_component parent);
     super.new( name, parent);
     // create the sequencer, driver and monitor
     monitor = new (“monitor”,this);
     if (is_active == UVM_ACTIVE)  begin
         driver = new(“driver”,this);
         sequencer = new(“sequencer”, this);
      end
   endfunction
endclass
```

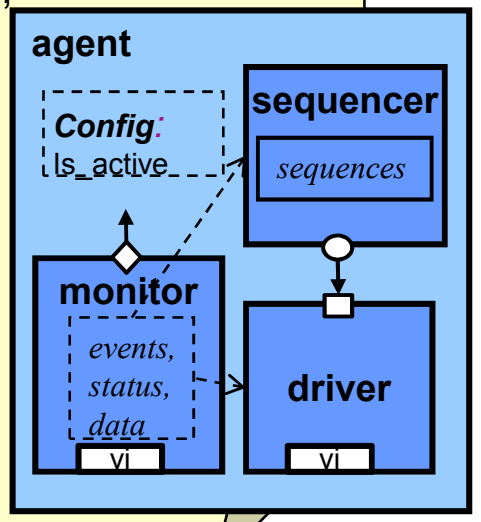
Derived from **uvm_agent**

Configuration field

UVM field declaration for library automation

Using the constructor to allocate the sub-instances is problematic because new() is not polymorphic

Allocation using new() forces a re-write when modifications are required



cadence

Creating the Agent Using build_phase() Method

```
class master_agent extends uvm_agent;
    active_passive_enum is_active = UVM_ACTIVE;
   `uvm_component_utils_begin(master_agent)
     `uvm_field_enum(active_passive_enum, is_active, ALL_ON)
   `uvm_component_utils_end
   // agent sub component instances
    master_driver          driver;
    master_sequencer       sequencer;
    master_monitor         monitor;
  function new(input string name, input uvm_component parent);
      super.new(name, parent);
  endfunction
  virtual function void build_phase(uvm_phase phase);
    // Allocate sequencer, driver and monitor here
  endfunction
 virtual function void connect_phase(uvm_phase phase);
    // Make TLM connections here
  endfunction
endclass
```

Constructor is still required

Use the **build_phase()** method to allocate components and address the polymorphism requirement

Use the **connect_phase**() method to make TLM connections between the sub-components

cadence™

The Agent build_phase() and connect_phase()

```
class master_agent extends uvm_agent;
  . . .
  // agent sub component instances
  master_driver          driver;
  master_sequencer   sequencer;
  master_monitor       monitor;
  . . .
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(uvm_phase phase);
  // create sequencer, driver and monitor
    monitor = master_monitor::type_id::create( “monitor”, this);
    if (is_active == UVM_ACTIVE)  begin
      driver = master_driver::type_id:: create(“driver”, this);
      sequencer = master_sequencer::type_id::create(“sequencer”, this);
    end
  endfunction
  virtual function void connect_phase(uvm_phase phase);
      driver.seq_item_port.connect(sequencer.seq_item_export);
   endfunction
endclass
```

```
class v2_mdriver extends master_driver;
    // add attributes + override virtual methods
endclass
// syntax for introducing the new driver – in test
master_driver::type_id::set_type_override (
      v2_mdriver::get_type());
```

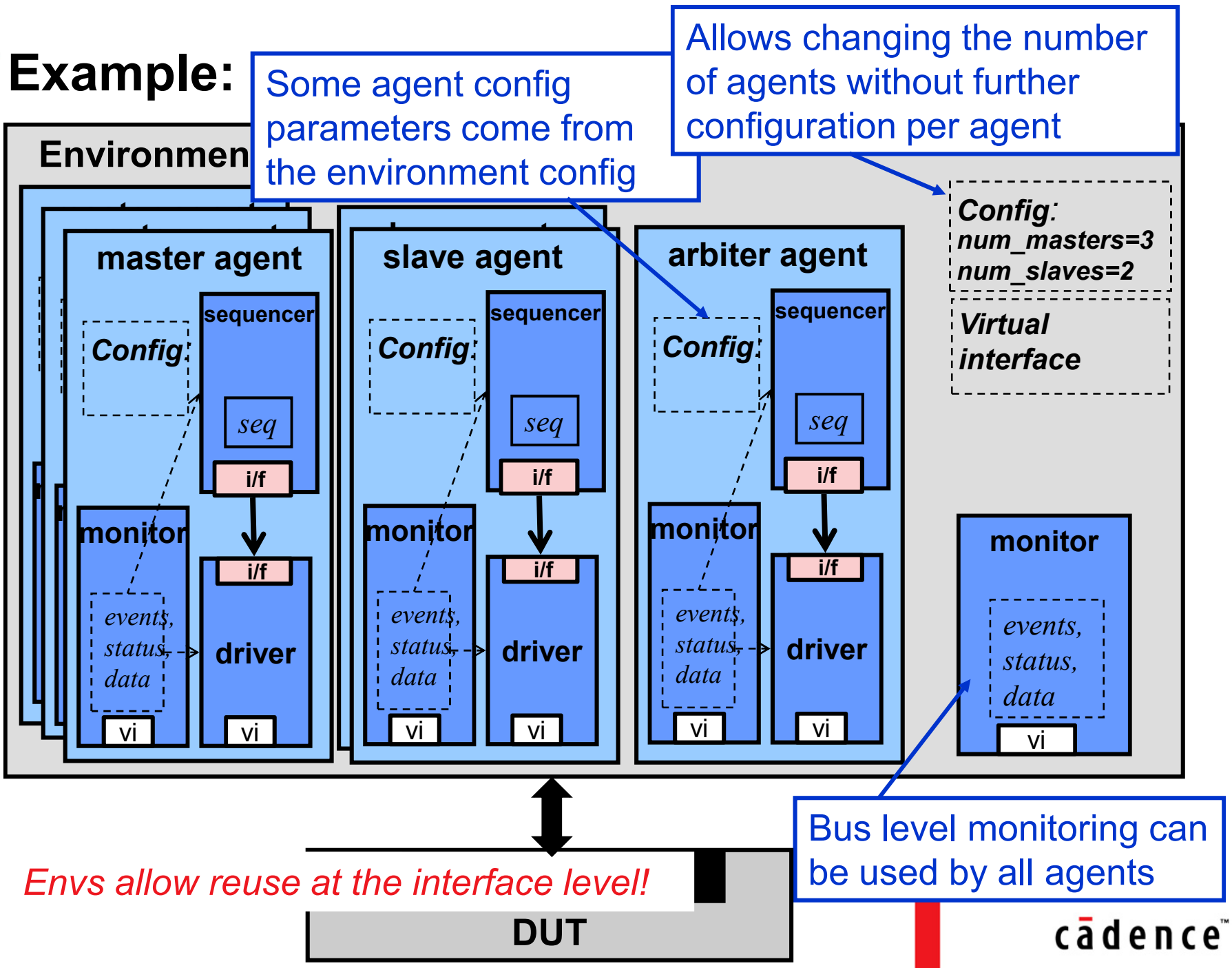
Use the factory to override the driver for a specific test

Call super.build_phase(…)

Use **create()** for factory allocation

Standard TLM mechanism for connecting the driver and sequencer

Example:



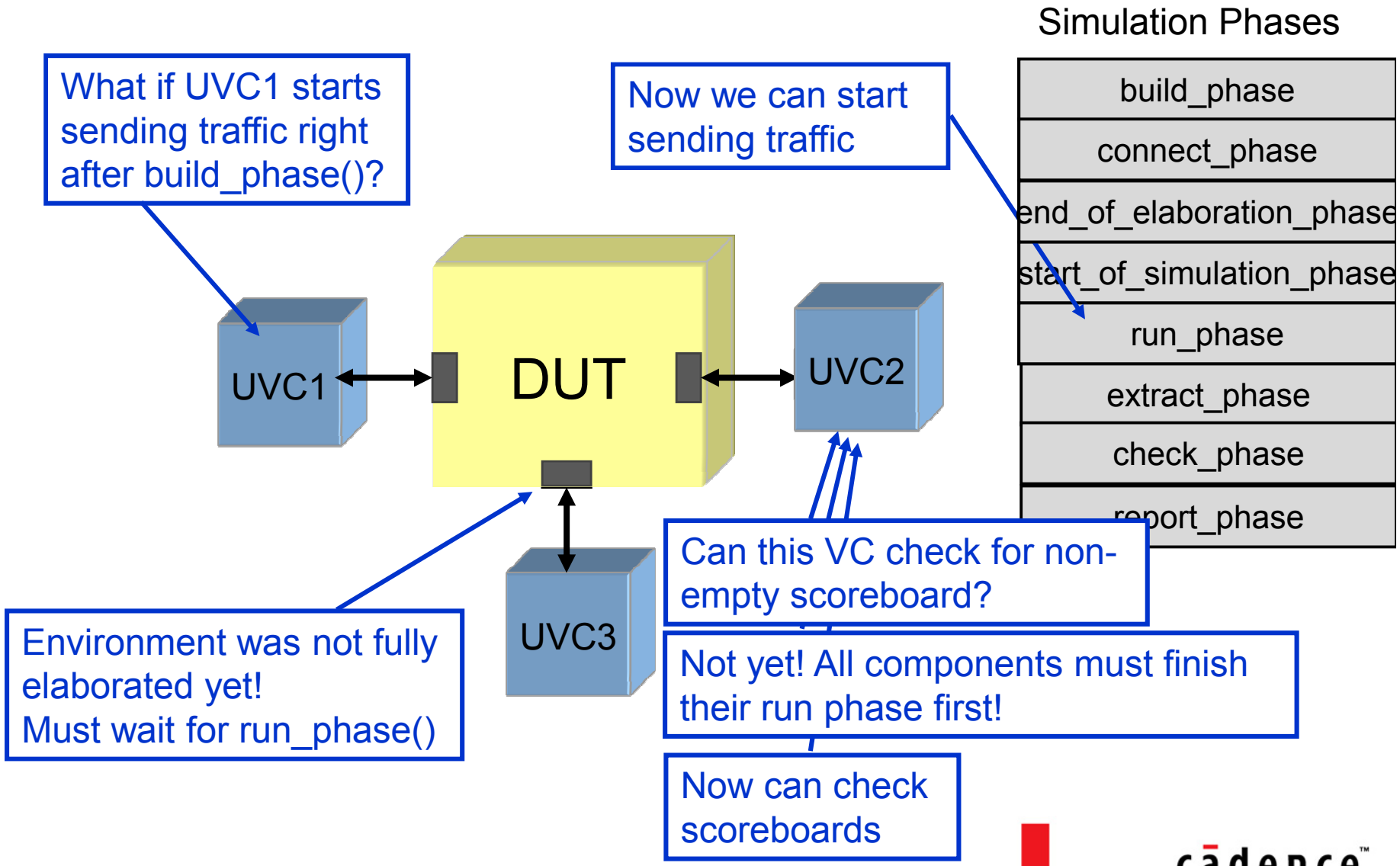
cadence

Simulation Phases and Coordination

- When moving to classes you need to manage environment creation at run-time …
- Test execution is divided to phases
  - Configuration, testbench creation, run-time, check, etc
- Unique tasks are performed in each simulation phase
  - Set-up activities may be performed during “testbench creation” while expected results may be addressed in “check”
  - Phases run in order – next phase does not begin until previous phase is complete
- A set of *standard* phases enables VIP plug & play
  - Allows orchestrating the activity of components that were created by different resources

cadence™

Simulation Phases and Coordination



cadence™

UVM Simulation Phases

UVM component's built-in phases – run in order

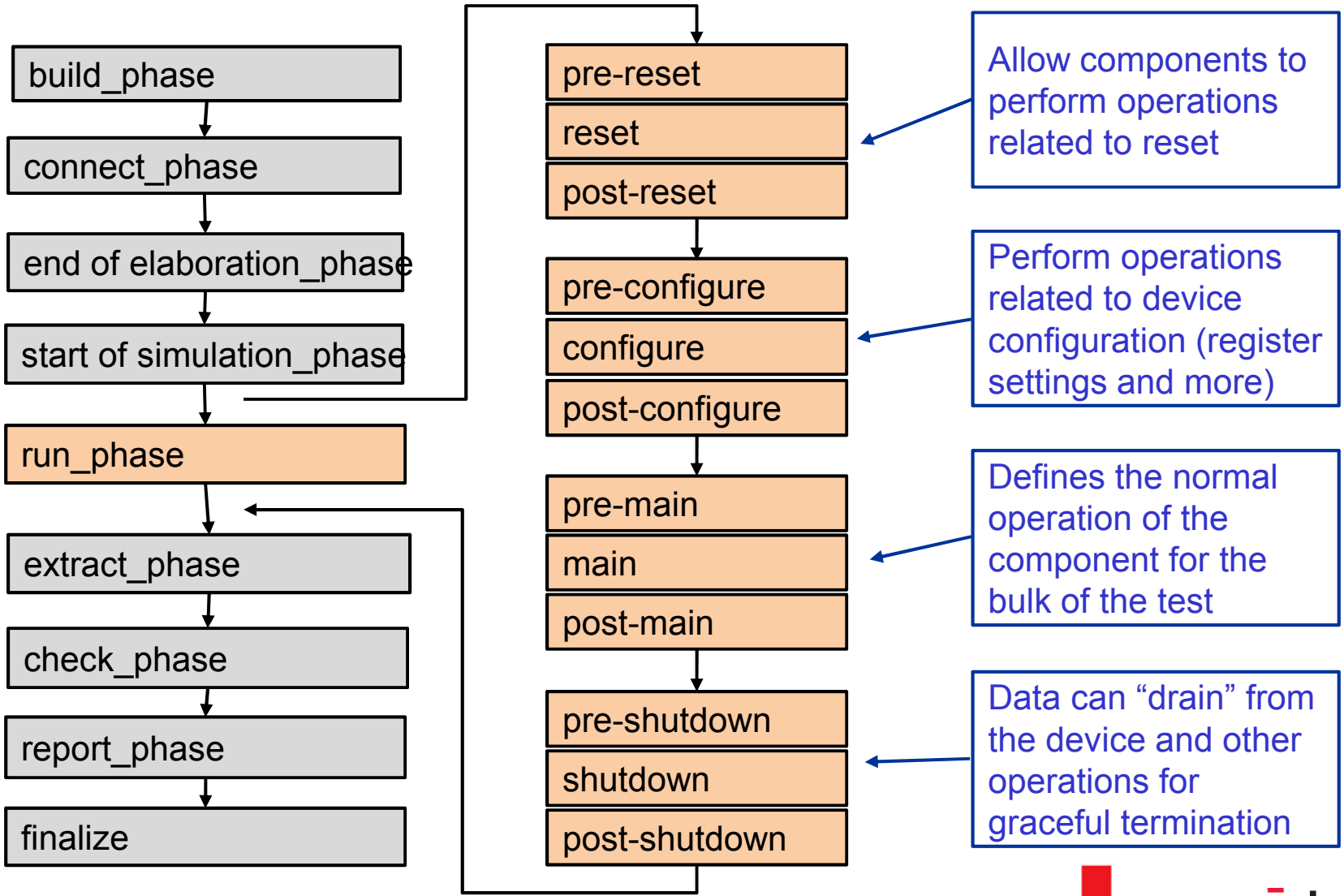
All phases except run_phase()
Execute in zero time

build_phase	Build Top-Level Testbench Topology
connect_phase	Connect environment topology
end of elaboration_phase	Post-elaboration activity (e.g. print topology)
start_of_simulation_phase	Configure verification components
run_phase	task – Run-time execution of the test
extract_phase	Gathers details on the final DUT state
check_phase	Processes and checks the simulation results
report_phase	Simulation results analysis and reporting

© 2011 Cadence Design Systems, Inc. All rights reserved.
cadence®

UVM Run-Time Phases

Time consuming and run in parallel to run_phase



© 2011 Cadence Design Systems, Inc. All rights reserved.

cadence®

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

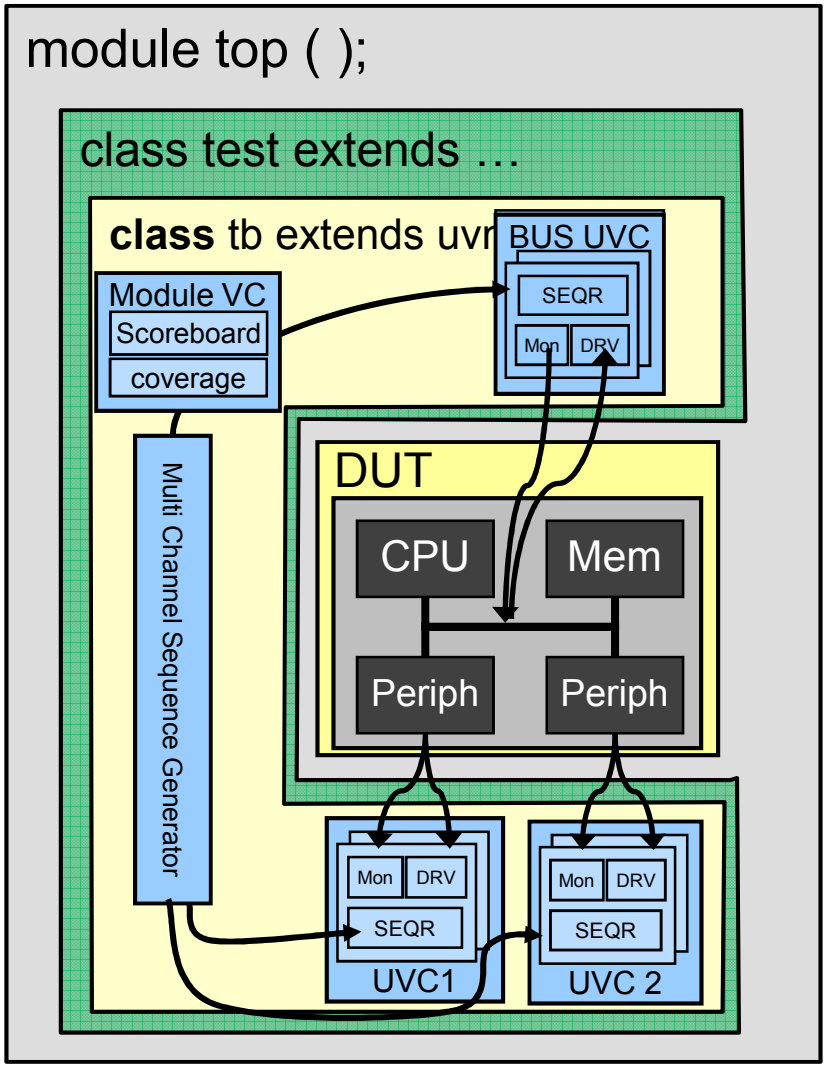
- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components
  - Controlling verification environment
  - Automating environment creation with IVB

UVM Multi-Language and UVM Acceleration

Summary

cadence™

Testbench Environment (TB)

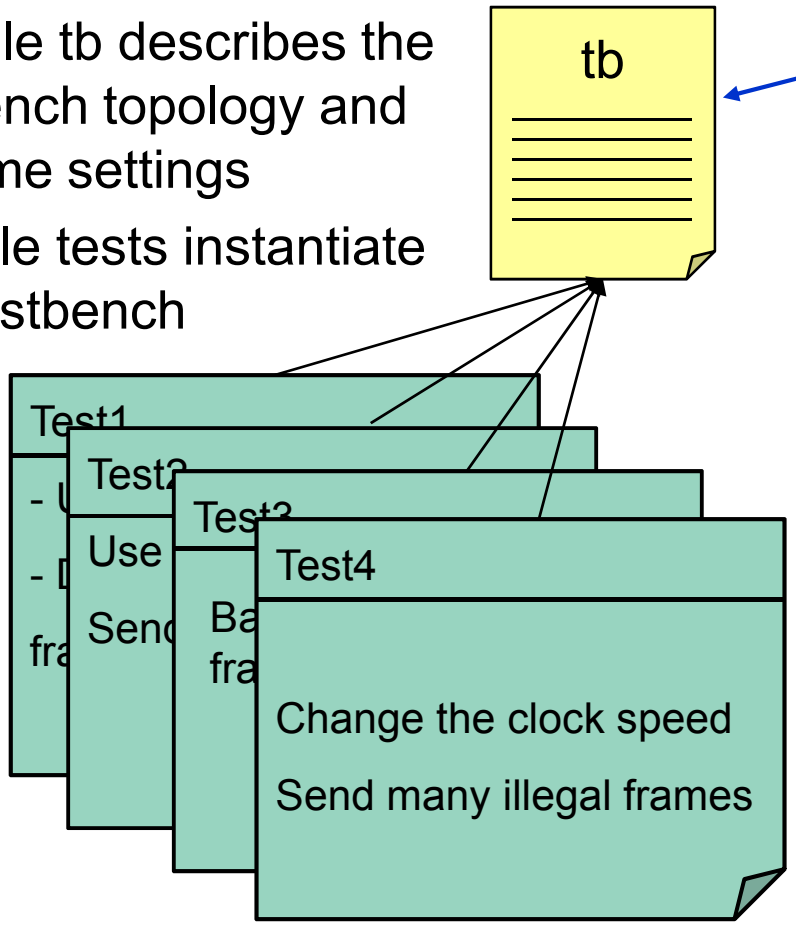


- Separate the env configuration and the test
  - TB class instantiates and configures reusable components
- Specific tests are built on top of testbenches (tbs)
  - Specify the nature of generated traffic
  - Can modify configuration parameters as needed
- Benefits
  - Tests are shorter, and descriptive
  - Less knowledge to create a test
  - Easier to maintain – changes are done in a central location

cadence

Using a tb for Multiple Tests

- A single tb describes the testbench topology and run-time settings
- Multiple tests instantiate this testbench

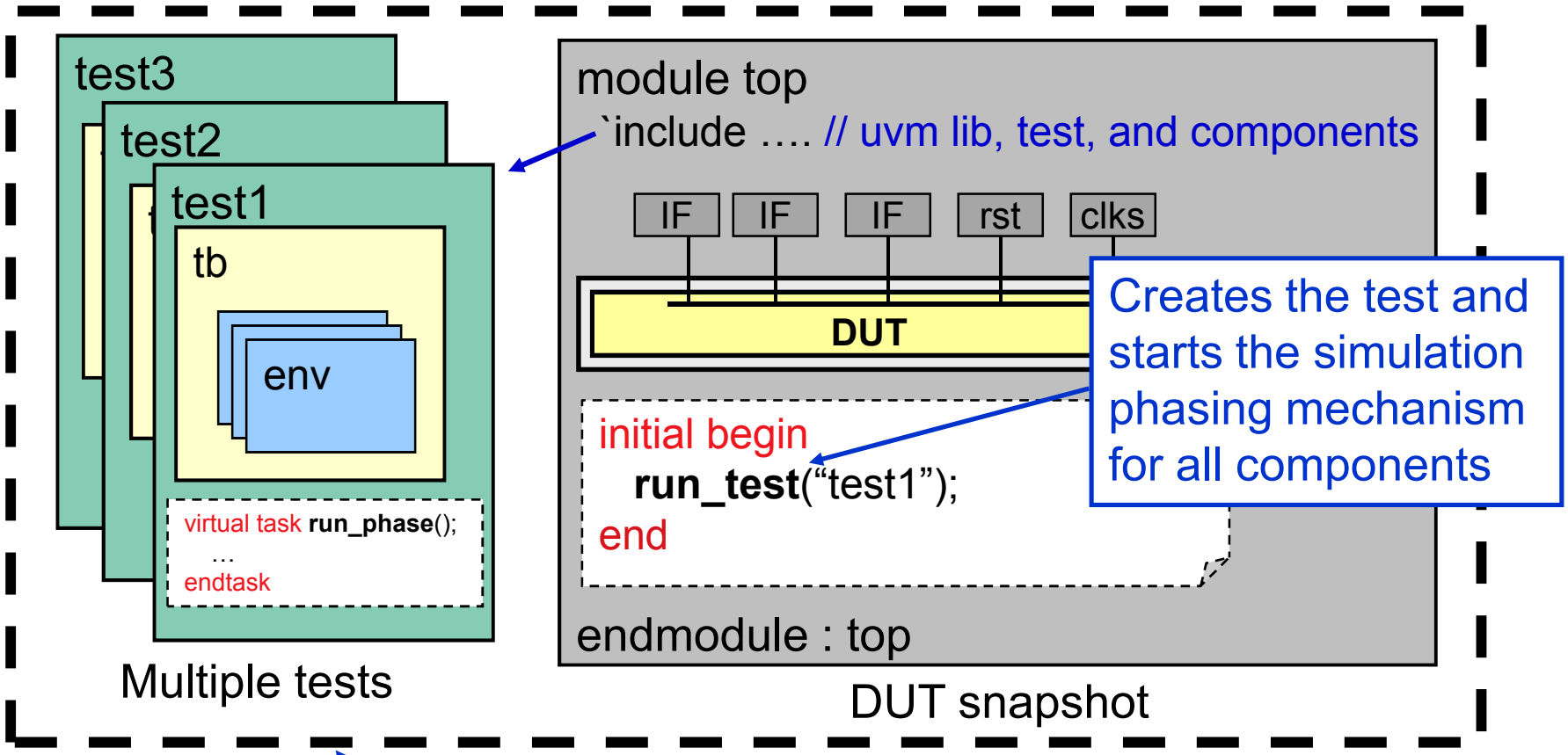


If config needs to be changed, no need to visit all tests

- Tests determine the nature of the generated traffic

cadence

The Test Launching Mechanism



Testbench Example

```
class uart_ctrl_tb extends uvm_env;
  `uvm_component_utils(uart_ctrl_tb)

 apb_env apb0;    // APB UVC
  uart_env uart0;    // UART UVC
  uart_ctrl_env uart_ctrl0;  // Module UVC

  virtual function void build();
        super.build();
        uvm_config_db#(uvm_bitstream_t)::set(this,“a
     “is_active”, UVM_ACTIVE);
        uvm_config_db#(uvm_bitstream_t)::set(this,“uart0.tx”, “is_active”,
     UVM_ACTIVE);
        uvm_config_db#(uart_ctrl_config)::set(this,“uart_ct
        apb0 = apb_env::type_id::create(“apb0”, this);
        uart0 = uart_env::type_id::create(“uart0”, this);
        uart_ctrl0 = uart_ctrl_env::type_id::create(“uart_ctrl0”, this);
    endfunction
endclass
```

Extends from uvm_env

Configure using wildcards

Minimal effort and component knowledge required

Create and build using standard mechanism

cadence™

How a Test Uses a UVM Testbench?

```
class apb_uart_rx_tx extends uvm_test;
 `uvm_component_utils(apb_uart_rx_tx)

 uart_ctrl_tb  uart_ctrl_tb0;    //testbench
virtual function void build_phase(uvm_phase pha
   super.build_phase(phase);
   // Set the default sequence for master
uvm_config_db#(uvm_object_wrapper)::set(this,“uart_ctrl_tb0.apb0.master.
     sequencer.run_phase",“default_sequence",
     "read_modify_write::type_id::get()");
   //Construct  and build the uart_ctrl_tb0 environment
   uart_ctrl_tb0 = uart_ctrl_tb::type_id::create(“uart_ctrl_tb0”, this);
 endfunction: build_phase

virtual task run_phase(uvm_phase phase);
   uvm_top.print_topology();
    …
  endtask
endclass: apb_uart_rx_tx
```

Extends from uvm_test

Configuration information specific for this test

Test creates and builds the testbench

run() task is optionally used for run-time test control or status. e.g., print dynamic testbench topology

cadence™

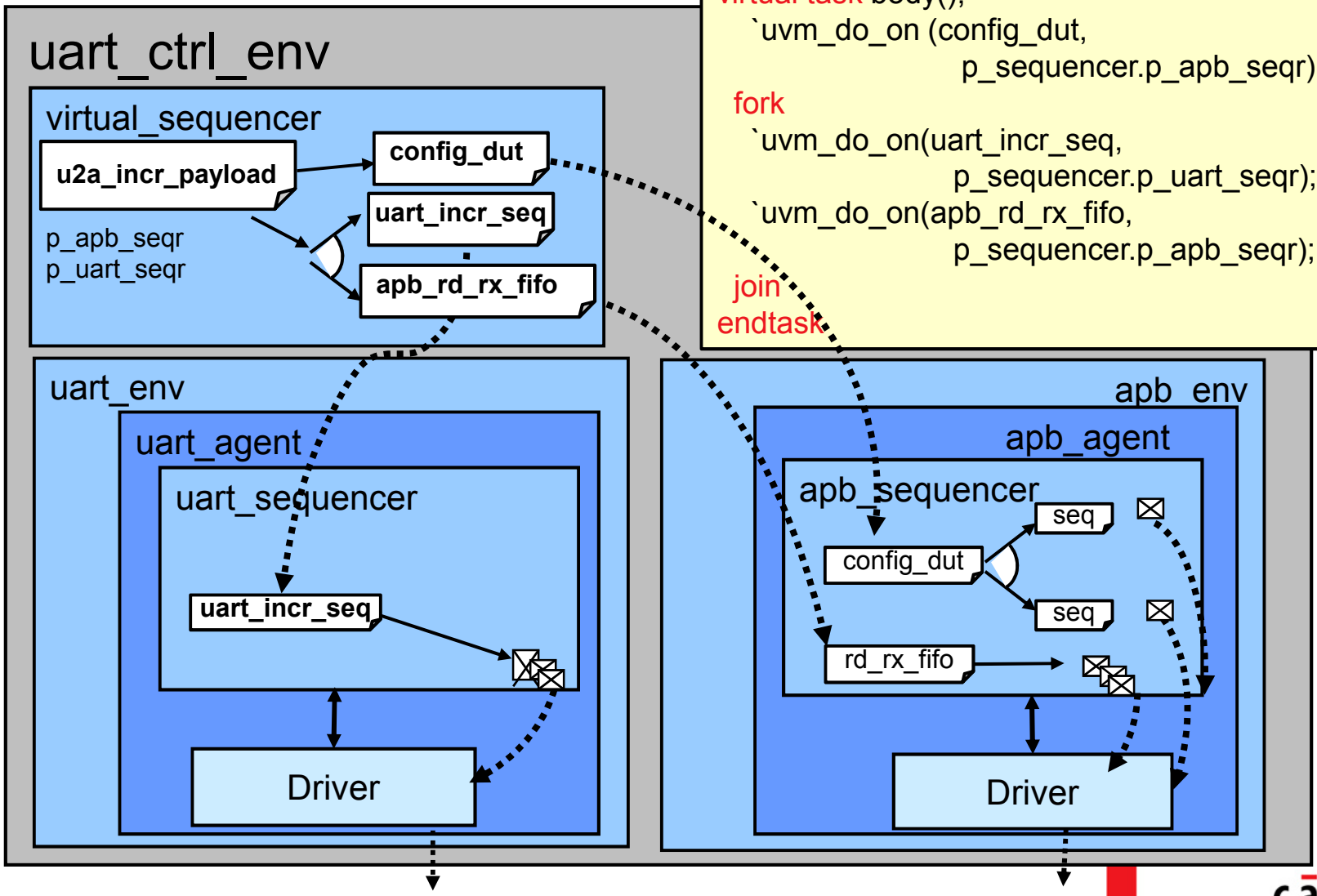
Multi-Channel Sequences

- So far, we have seen how to control stimulus for a single channel (interface)
  - Create a sequencer
  - Create and register sequences
- In a verification environment, we need to coordinate traffic on multiple interfaces in parallel
  - Coordinate data and time
  - “After finishing configuring the device send Ethernet traffic”
- Multi-channel sequences are called virtual sequences
  - Single procedural thread to control multiple interfaces
  - Can create reusable system-level sequences

cadence™

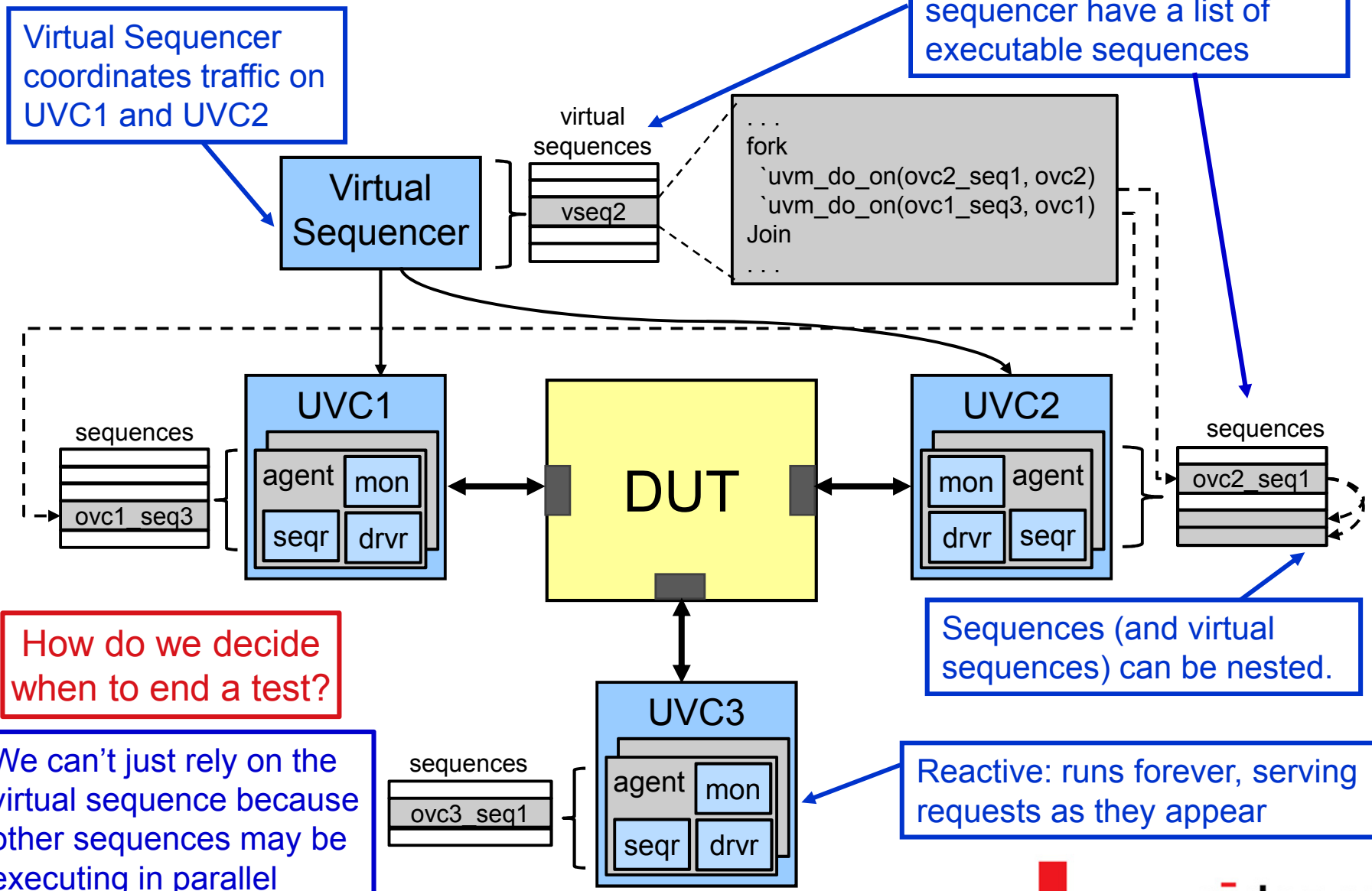
Multi-Channel Sequences

```
class u2a_incr_payload extends uvm_sequence;
 . . .
virtual task body();
   `uvm_do_on (config_dut,
                    p_sequencer.p_apb_seqr);
  fork
   `uvm_do_on(uart_incr_seq,
                    p_sequencer.p_uart_seqr);
   `uvm_do_on(apb_rd_rx_fifo,
                    p_sequencer.p_apb_seqr);
  join
endtask
```



cadence

Determining End-of-Test



cadence

Raising and Dropping Objections

```
class retry_seq extends uvm_sequence#(uart_frame);
  ...
  task body();
    `uvm_info(get_type_name(), “Executing retry_seq", UVM_LOW)
    starting_phase.raise_objection(this, “retry_seq");
    `uvm_do_with(req, {payload == pload; parity == BAD_PARITY;} )
     ...
    starting_phase.drop_objection(this, " retry_seq ");
   endtask
endclass
```

- `starting_phase` is a pointer to the phase in which the sequence is executed
- Objections are handled hierarchically
  - Every component has a objection count (`phase_done`) for every run phase
  - Both raised and dropped objections propagate up the hierarchy
  - When the `uvm_top` objection count for a phase returns to zero, the phase can end
- Not all sequences should raise objections
  - Not appropriate for background traffic, response generators, sub-sequences etc.

cadence™

Drain Time

- When all raised objections are dropped, simulation is stopped
- A drain time allows propagation of outstanding items (e.g., DUT response)
  - If no drain time is set, the test will stop immediately
- Drain time can be set as:
  - Absolute delay by calling `set_drain_time` for a specific phase
    - Usually from a test class

```
task run_phase(uvm_phase phase);
. phase.phase_done.set_drain_time(this, 200ns);
endtask : run_phase
```

  - Event delay by defining method `all_dropped` in env or other component
    - Automatically called when objection count for component returns to 0.

```
task all_dropped (uvm_objection objection, uvm_object source_obj,
                  string description, int count)
```


cadence™

Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components
  - Controlling verification environment
  - Automating environment creation with IVB

UVM Multi-Language and UVM Acceleration

Summary

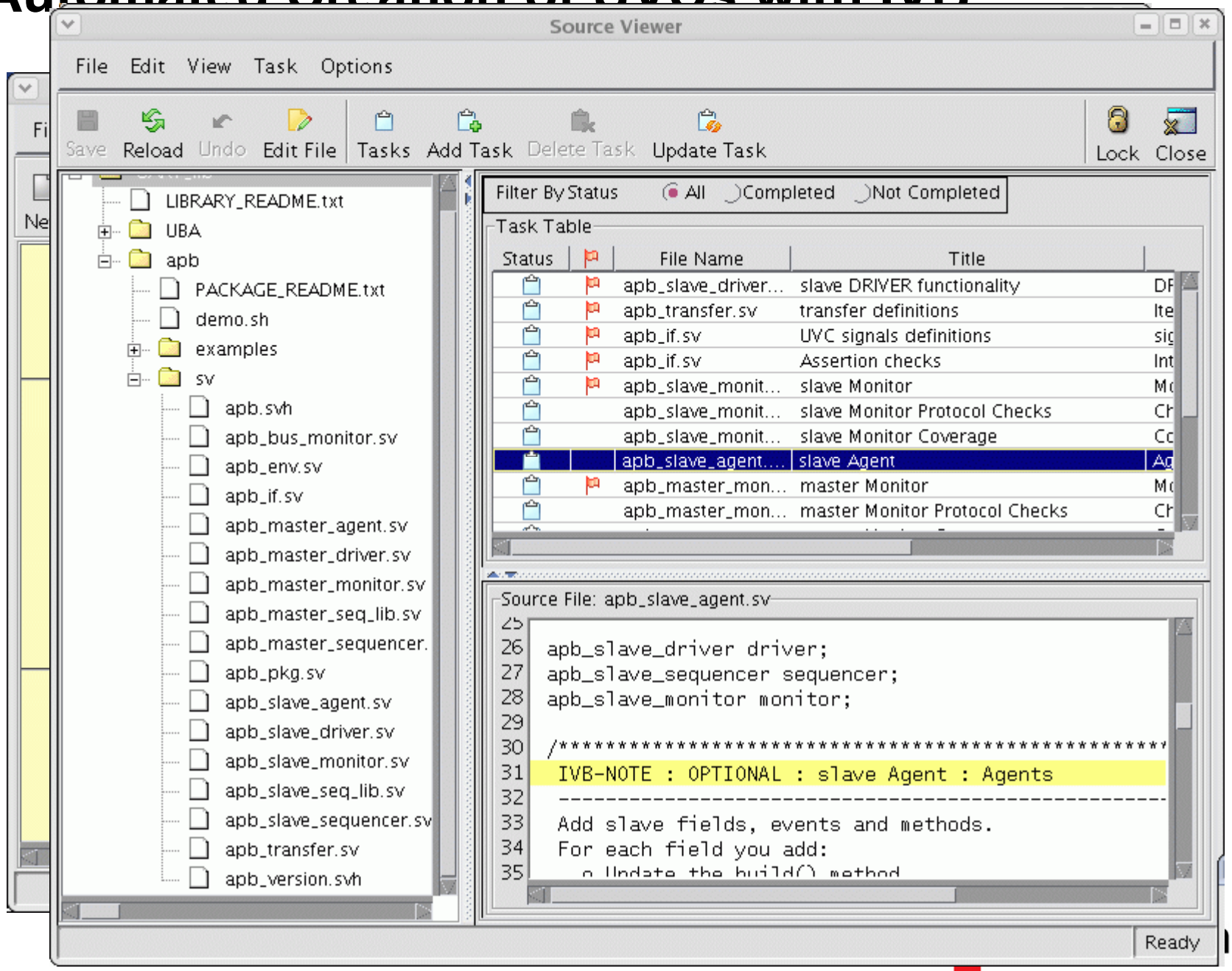
cadence™

IVB – About the Incisive Verification Builder What is IVB ?

- Incisive Verification Builder (IVB) is a GUI-based tool
  - Automates the process of creating reusable multi-language verification environments from the interface-level to the system-level
- Creates UVM-compliant Verification Component skeleton
  - UVC architecture, including agents, data-items, sequencers, driver, monitor, tb, test
- Specifies (via NOTES) where to add protocol-specific implementation code
- Produces a working example on how to use the UVC in a simulation environment

cadence™

Automated Creation of UVCs with IVB



Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

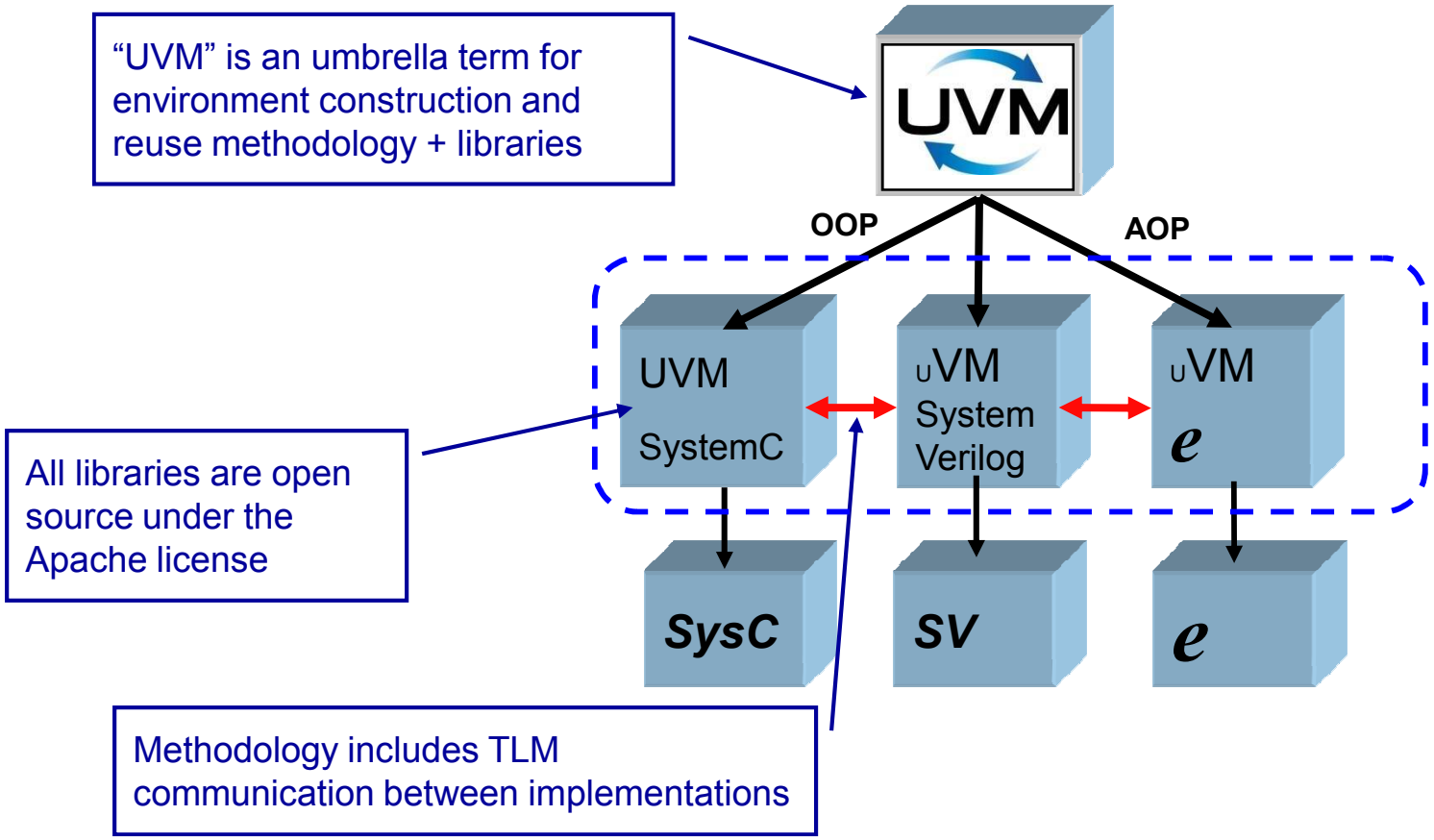
- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

cadence™

UVM Multi-Language (ML) – Enable Mixed-Language Env

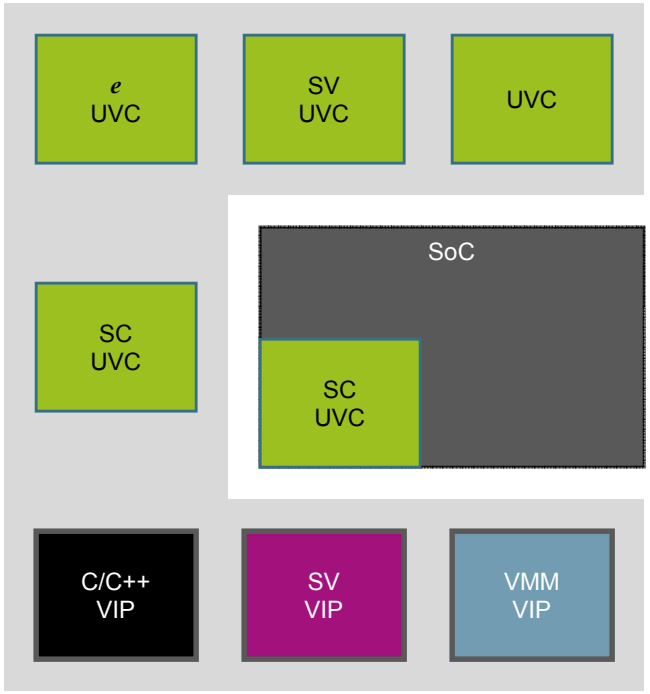


- **Proven methodology independent of language**
- **Built-in language interoperability**

cadence

UVM Enables Mixed-Language Verification

Reduces risk and unifies SoC verification teams

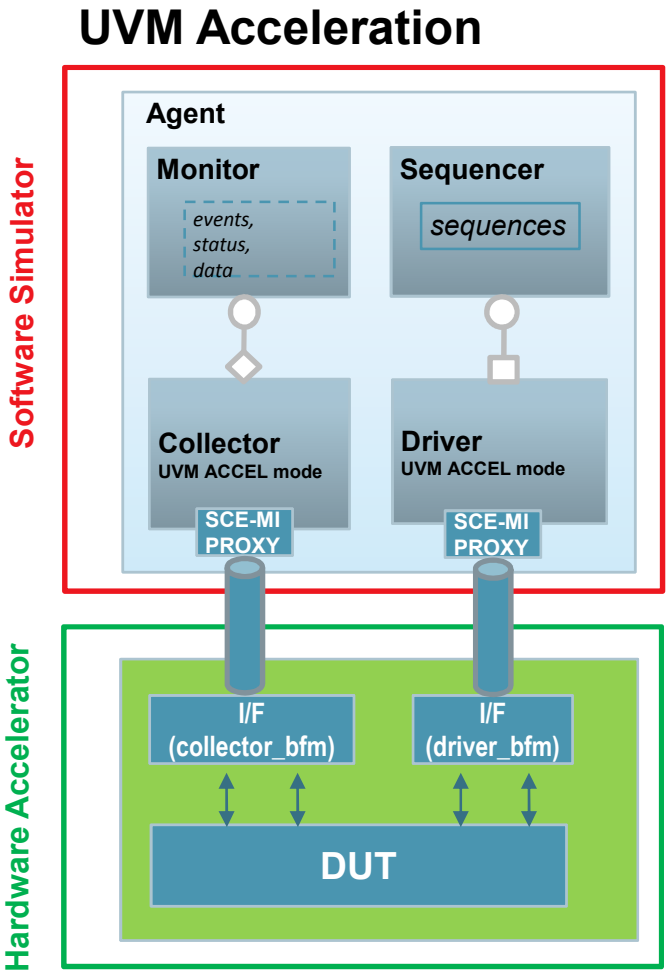


- Single UVM methodology for SystemVerilog, *e* and SystemC
- Unifies VIP for reuse
- Choose best language for application and group skills
- Enables companies to choose best verification solution

cadence™

UVM for acceleration

Extending agents for acceleration

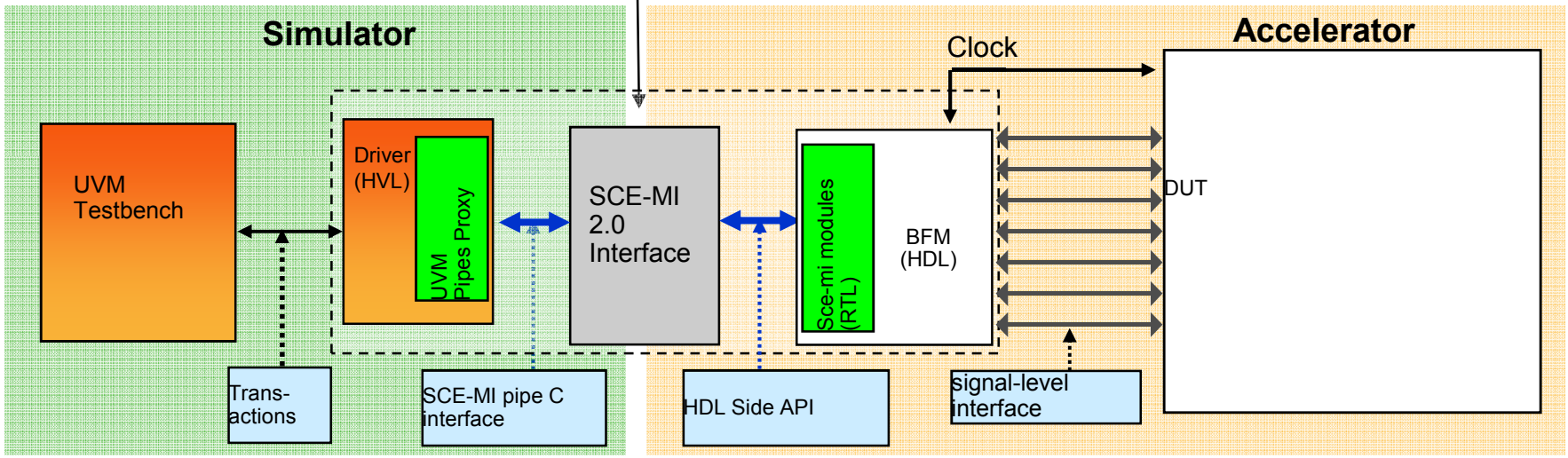


- Cadence provides
  - UVM library extensions: UVM_Accel
  - Methodology
  - Examples
- Support SV and e
- Enables verification re-use between simulation and acceleration

cadence

The Software – HDL Interface

SCE-MI 2.0 modules
- scemi_input_pipe
- scemi_output_pipe





Host Workstation

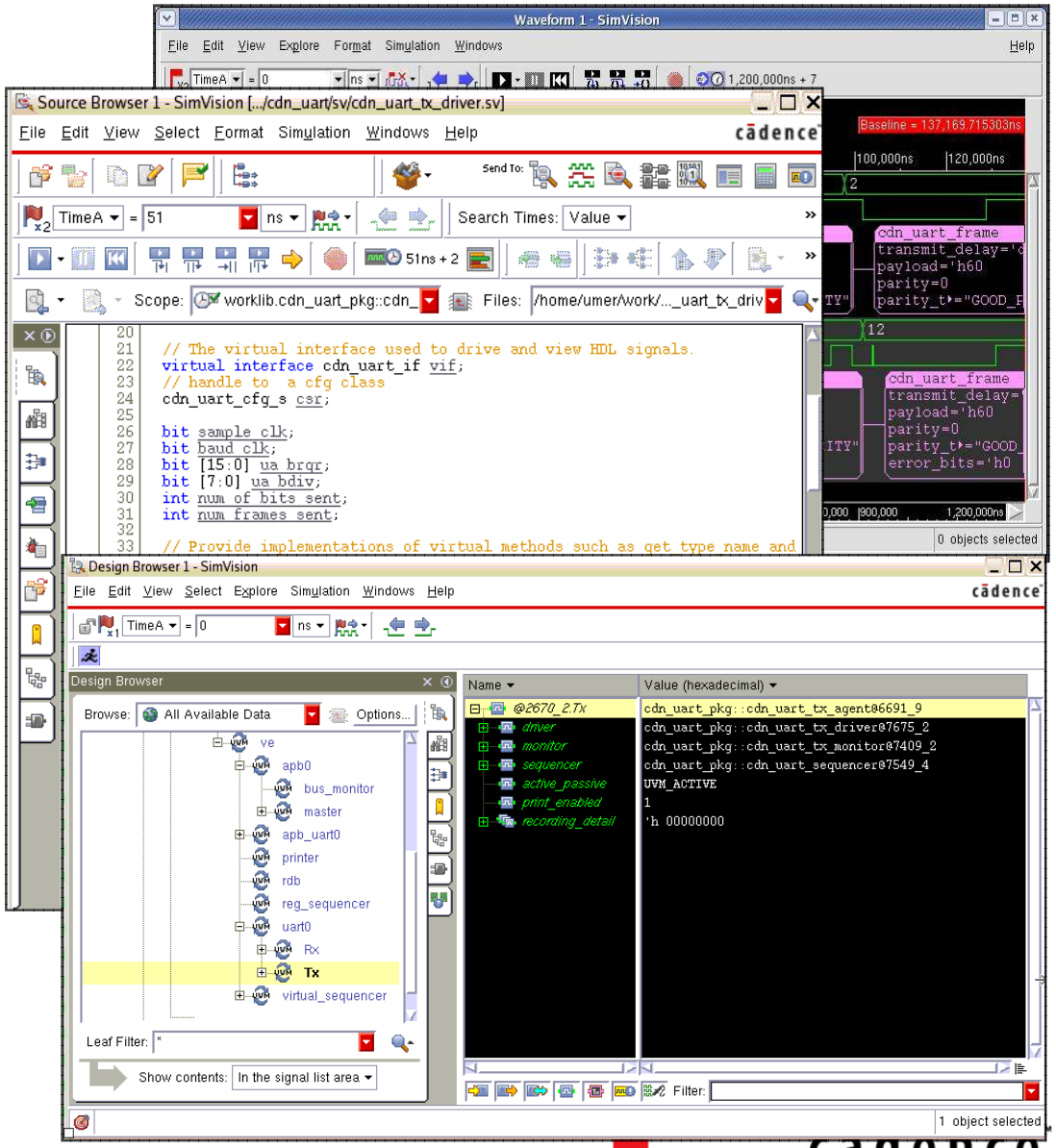
Cadence PdXP



cadence™

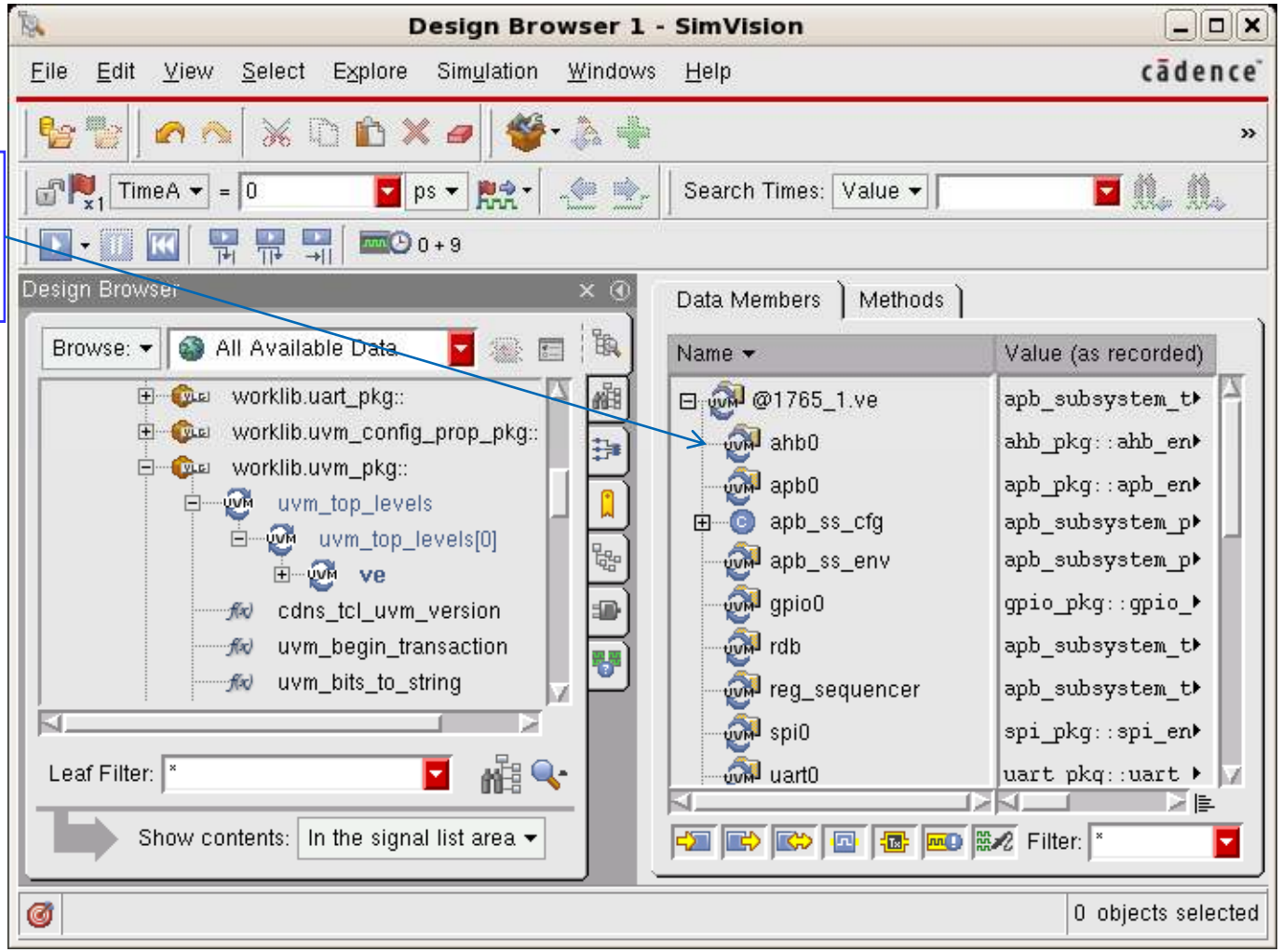
SimVision - Advanced UVM Analysis and Debug

- UVM-aware dynamic object debug
  - Debug testbench configuration and hierarchy
  - Set breakpoints
    - Step in, step through classes
  - Flexible TCL to manipulate testbench at run-time
- UVM transaction analysis
  - UVM transaction recording
  - Stripe chart
- Debug constraint conflicts
  - Identify over-constrained issues
  - Interactively reduce constraints and randomize on the fly



Design Browser

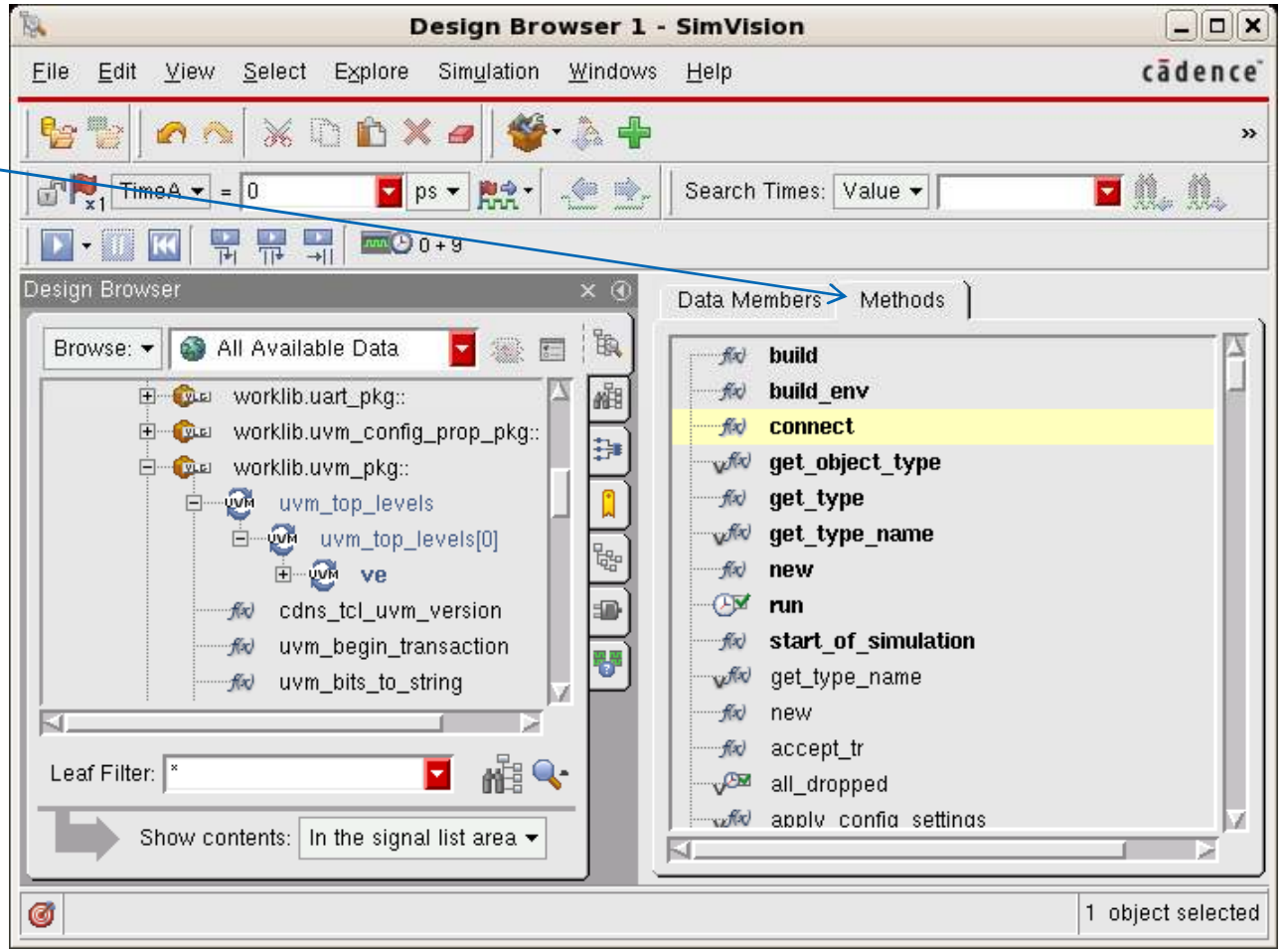
Icon for UVM data members Click on icon will “route” to Design Browser



cadence

Methods Tab

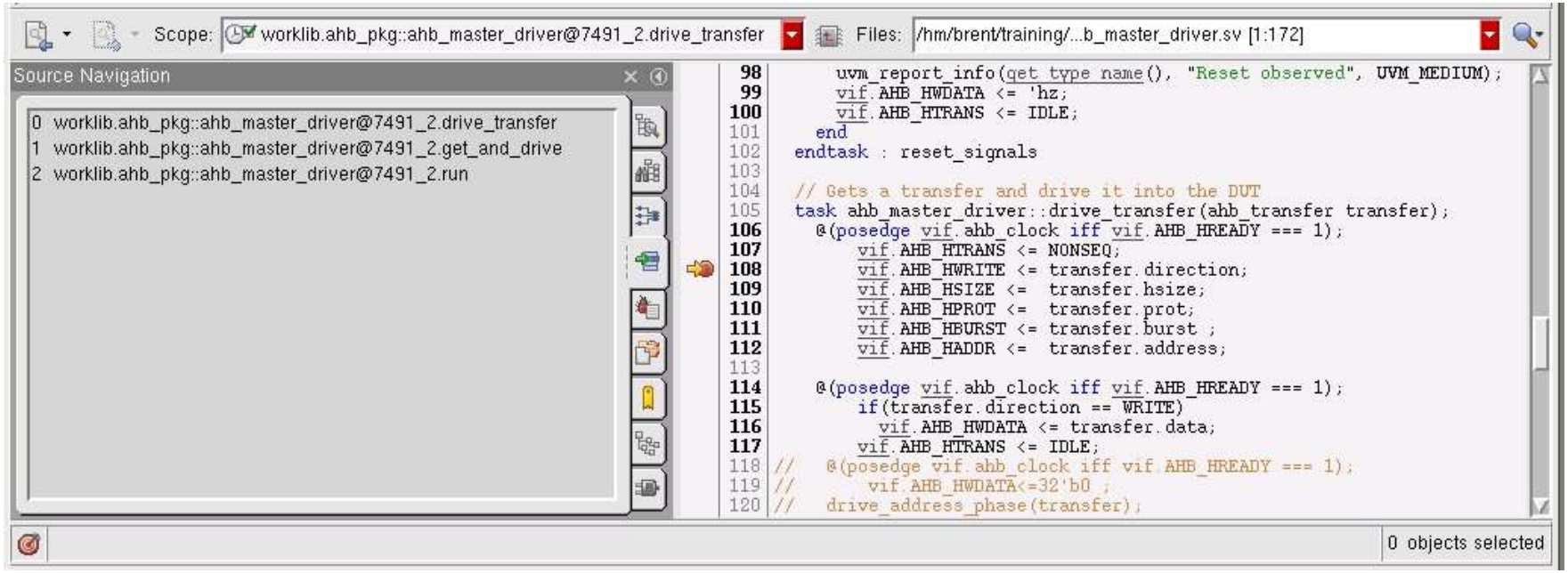
Methods Tab of the displayed UVM Component



cadence

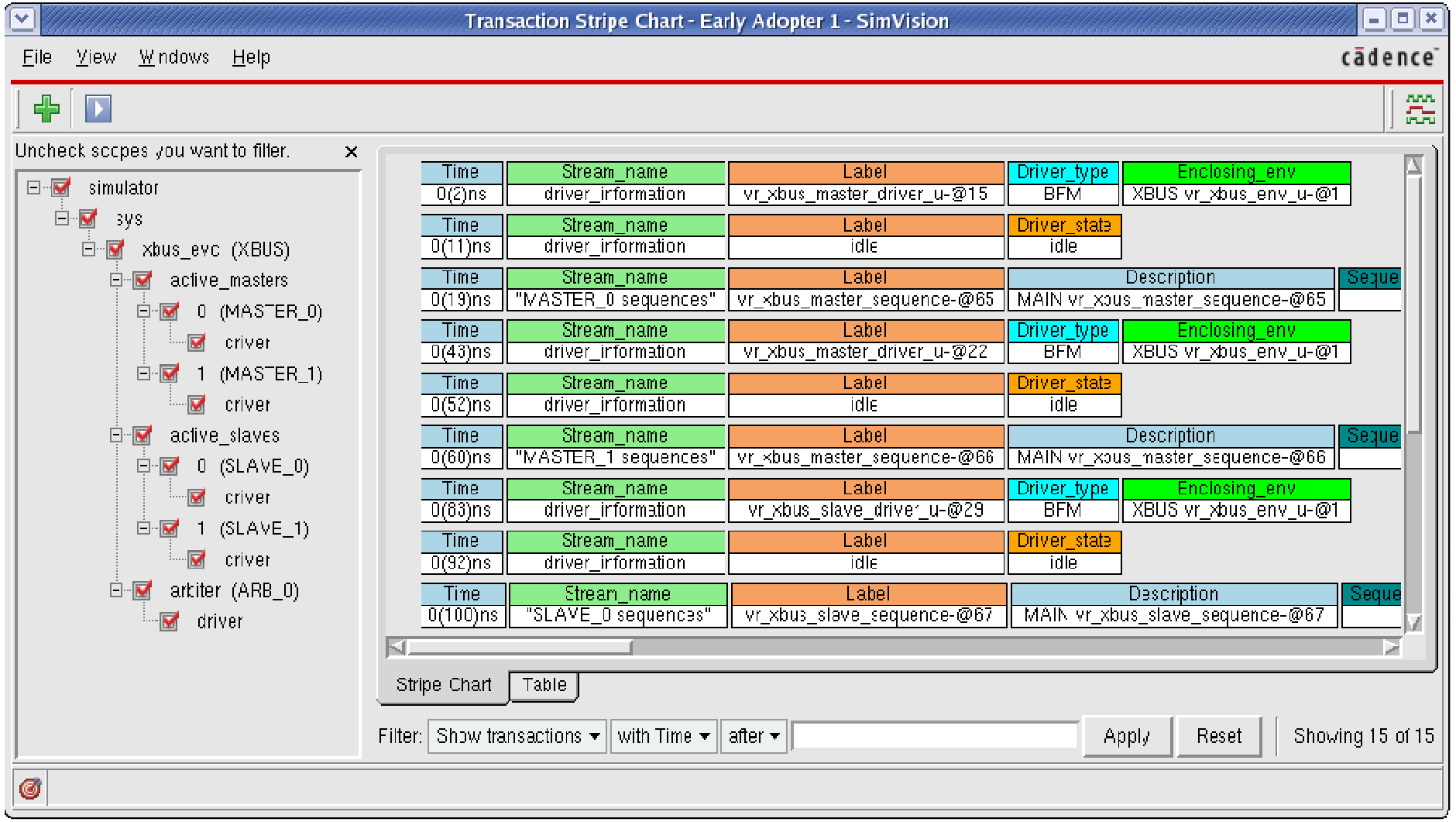
Interactive Sequence Debug Callstack Viewing

- When stopping a breakpoint, you can use the “callstack” sidebar to see where your method was called from and to traverse the callstack.



cadence™

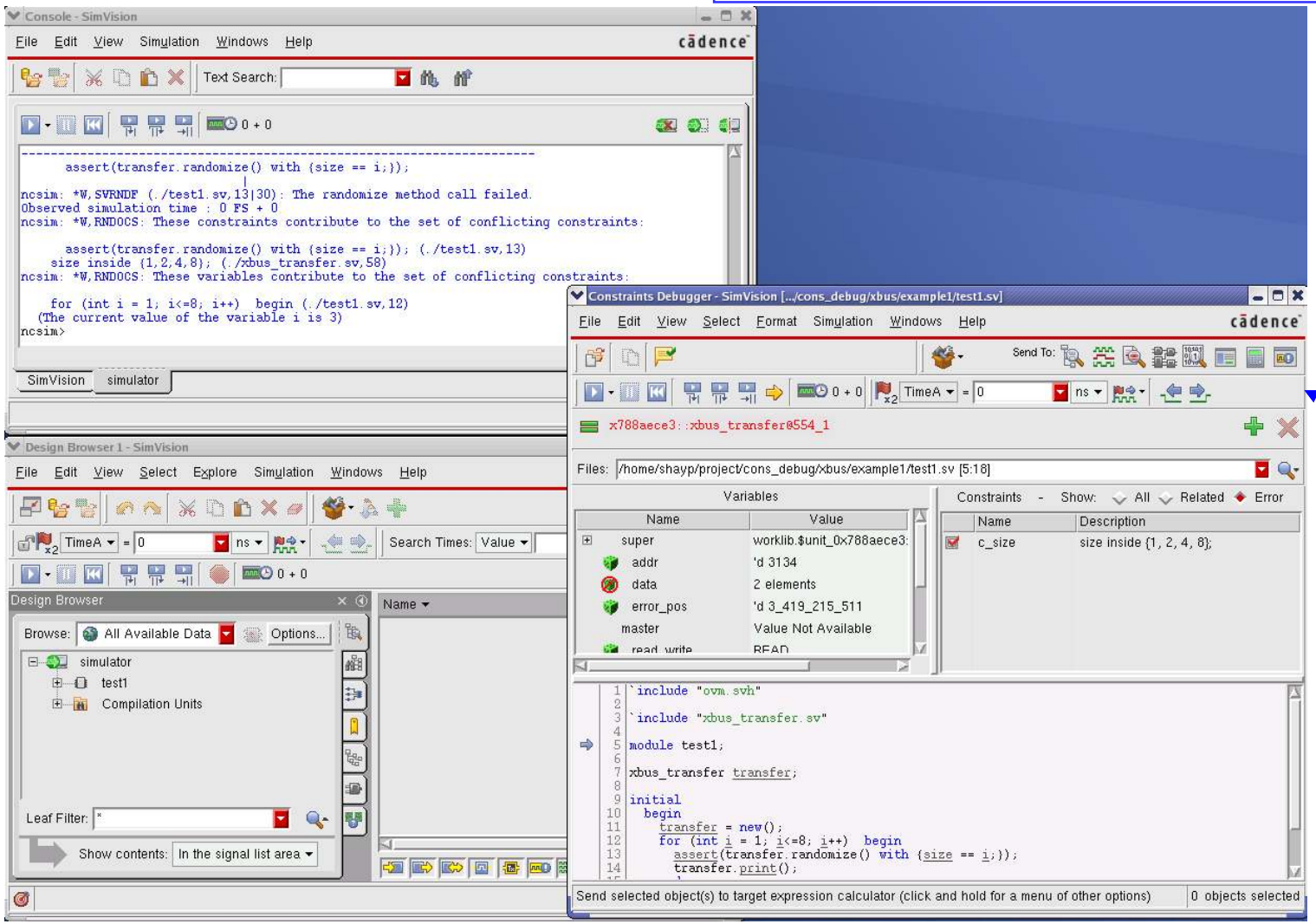
Transaction Stripe Chart (TSC)



cadence

Constraint Debugger

When there is a constraint solver failure, a new constraint debugger window is created for showing random variables and constraints associated with the class instance.

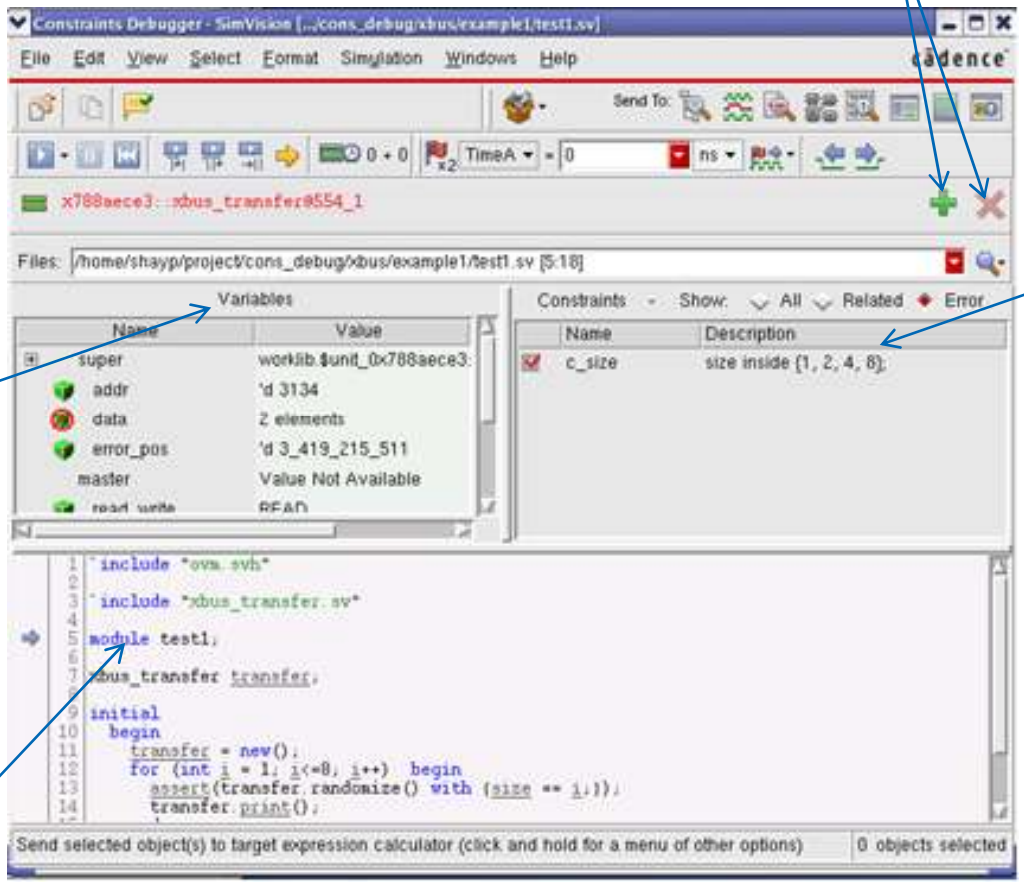


cadence

Constraint Debug

At the top of the window is a toolbar which has buttons that allow the user to create or delete a new constraint.

This portion lists the variables and their values. All variables are listed in the object since state variables can be used in a constraint.

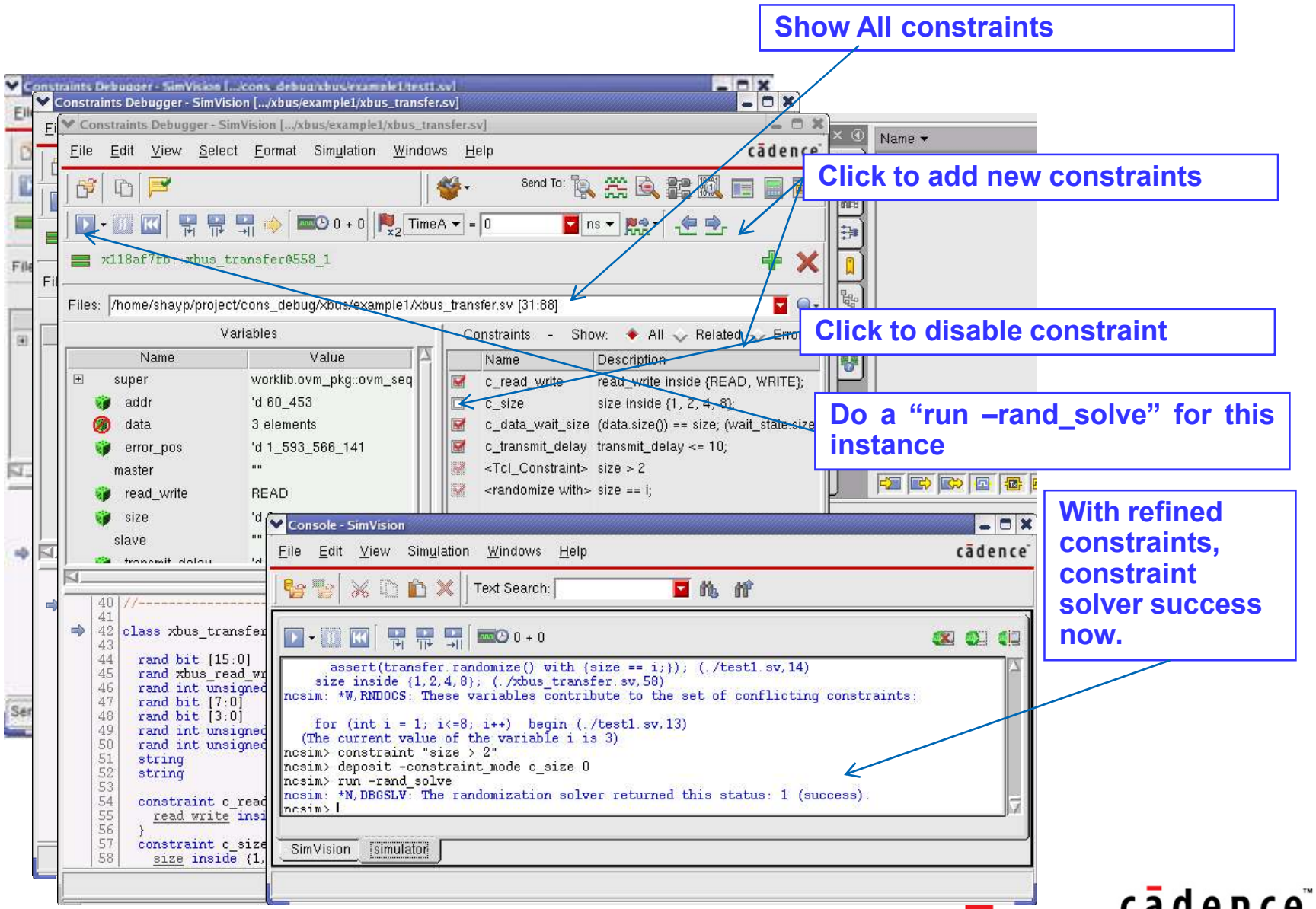


This portion lists the constraints for the object. This displays the constraint name and the first line of the constraint from the source file.

Source view region will show the selected rand variables and constraints in context.

cadence

Constraint Debug – Dynamic Analysis



Agenda

Introduction to UVM and Coverage Driven Verification

UVM:

- The UVM Library
- Stimulus Generation
- Building Reusable Verification Components
- Testbench Creation Using Reusable Components

UVM Multi-Language and UVM Acceleration

Summary

cadence™

What Have We Seen Today?

- Introduction to coverage-driven verification concepts
- Using UVM methodology and the UVM library to:
  - Accelerate CDV environments creation
  - Build reusable environments (and not only reusable classes)
- Ways to automate the verification process and achieve plan-driven verification
- Deploying the concepts and technology above allow you to:
  - Significantly shorten your verification cycle
  - Get better leverage of your entire team
  - Improve the overall quality of your DUTs

cadence™

Beyond UVM …
Inside the Incisive Platform

- Unique Metrics Driven Methodology around UVM
- Scalable register & memory modeling package
  - reg_mem downloadable from uvmworld.org/contributions.php
- Only proven plan-based verification management solution
- VIP plug & play with module-based SystemVerilog and *e*RM
  - Mixed-language methodology and automation
- Best simulation, formal, and hardware verification engines
- Wizards for environment creation (IVB)
- Transaction level dynamic environment debug

Over 200 worldwide verification Applications Engineers with Plan-to-Closure and methodology expertise to ensure successful project deployment

cadence™

Useful Websites…

- UVM World Website:

  http://uvmworld.org
  - Download latest version of UVM library, join the Community Forum to share ideas and have discussions with other UVM users
- Cadence User Community Website:

  http://www.cdnusers.org
  - Share ideas and have discussions with other Cadence tool users
- Cadence Educational Services Offerings:

  http://www.cadence.com/support/education/index.aspx
  - SystemVerilog Advanced Verification using UVM
  - SystemVerilog Language and Application

cadence™

Learning more…



- *A Practical Guide to Adapting the Universal Verification Methodology (UVM)*, by Sharon Rosenberg and Kathleen A Meade



- UVM Reference Flow

  http://uvmworld.org /uvm-reference-flow.php

  – An open-source contribution by Cadence Design systems, Inc.
  – Providing a UVM reference for users to train and learn
  – Enabling a uniform environment for executing UVM code
  – Establishing a standardized solution for benchmarking
  – Demonstrating the features provided with UVM Verification Components (UVCs)

cadence

Thank You!

